

Linear-Readout Reconstruction Floors and Support Recovery in Computation in Superposition

Hector Borobia^{a,*}, Elies Seguí-Mas^b, Guillermina Tormo-Carbó^c

^a*VRAIN – Valencian Research Institute for Artificial Intelligence, Universitat Politècnica de València, Valencia, Spain*

^b*Department of Economics and Social Sciences, Universitat Politècnica de València, Valencia, Spain*

^c*Department of Business Organisation, Universitat Politècnica de València, Valencia, Spain*

Abstract

Overcomplete neural representations encode more latent features than activation dimensions, but are useful only when that information can be decoded. Which interface a network maintains is decided informally today, and recent work disagrees about whether trained models compute in superposition at all. We give two computable criteria for a code, and prove that one implies the other. The first is analog reconstruction: a rank–trace bound makes any rank- d unit-diagonal readout carry average squared off-diagonal cross-talk at least $(F - d)/(d(F - 1))$, in a gain-normalised form bounding the interference-to-gain ratio. Since that constrains an ensemble rather than a code, we also compute a given code’s own floor in closed form, factoring any attainment ratio into a geometry term and a decoder term and showing that ratios reported under the pseudoinverse measure only the first. The second is support recovery, as a per-feature capacity: one linear programme returns the largest sparsity at which any affine probe can detect a chosen feature, with a certified noise margin when it succeeds and a checkable certificate when it fails. Low leverage then forces that frontier down, with no decoder in the argument, while an explicit code refutes the converse — so the hierarchy is strict and one-way. The statistics need $O(Fd^2)$ time and $O(d^2)$ memory, never the

*Corresponding author.

Email address: `hecboar@doctor.upv.es` (Hector Borobia)

$F \times F$ interface, reaching $F \approx 10^6$ features. Six CPU experiments validate the procedure; in a trained toy model the L^4 code’s near-attainment (1.0006 against 19.0052 for L^2) proves to be a property of its leverage profile rather than of its decoder.

Keywords: overcomplete representations, computation in superposition, linear readout, Welch bound, sparse recovery, neural network capacity

1. Introduction

Modern neural networks use high-dimensional activation spaces to encode many latent variables. In many settings the number of relevant latent features exceeds the ambient dimension, which motivates the study of *overcomplete representations*: a state in $\mathbb{R}^d$ carrying information about $F \gg d$ features.

For such representations the central issue is not storage. A code may passively embed many states, but a network must also decode what it needs and compose it across layers. Different decoding interfaces impose different constraints. A *linear readout* interface asks for a matrix G whose coordinates approximately recover the features. A *threshold* interface asks only that active and inactive coordinates be separated by a margin, so that a nonlinearity can recover a sparse Boolean state. These criteria are related but not equivalent, and the difference is not cosmetic: it decides which capacity results apply.

1.1. The concrete limitation

The distinction is currently drawn informally, and that has a cost. Hänni et al. [1] give an approximate-linear recursive template certifying $\tilde{O}(d^{3/2})$ computable features in width d , while Adler and Shavit [2] reach a near-quadratic regime using thresholded Boolean recovery. The two are routinely read as competing capacity claims. In parallel, an active empirical line disagrees about whether trained toy models compute in superposition at all: Bhagat et al. [3] argue that the standard L^2 -trained compressed-computation model largely exploits a mixing term, whereas Ferreira da Silva and Heimersheim [4] show that training under an L^4 loss elicits a solution that does compute in superposition. The disagreement turns precisely on which decoding interface the trained network maintains — and there is no standard, computable way to answer that question about a given network.

That is the gap this paper addresses. What is missing is not another capacity bound but an *operational test*: given a code or a trained model, decide which interface it supports and quantify how far it is from the best possible one.

1.2. The proposed method

We propose the **Interface Diagnostic** (ID), a procedure with a precise input, output and cost. Its input is an overcomplete code $\Phi \in \mathbb{R}^{d \times F}$ — hand-built, optimised, or read off a trained network — together with a sparse-state model and a decoding threshold. Its output is a triple:

1. the *attainment ratio* $\widehat{W}/W(F, d)$, where $\widehat{W}$ is the mean squared off-diagonal cross-talk of the calibrated readout interface and $W(F, d) = (F-d)/(d(F-1))$ is a floor no readout can beat (Theorem 5.1);
2. the *recovery profile* $p_{\text{rec}}(s)$ of the threshold decoder together with the exact per-coordinate reconstruction error that any linear readout must retain at the same s ;
3. a *criterion-separation witness*: the largest sparsity at which the support is recovered exactly with high probability while no rank- d linear readout can reconstruct the state’s real-valued coordinates to better than $\Omega(\sqrt{s/d})$ per coordinate.

A word on what the two criteria are, because the natural shorthand is misleading. The contrast is *analog reconstruction* against *support recovery*, not linear against nonlinear. Thresholding a linear score is itself a linear-threshold rule, and we evaluate exactly that rule alongside the network’s own output; the floor does not forbid it from recovering supports. What the floor forbids is small real-valued error. The two criteria therefore come apart because they ask different questions of the same map — “what is the value of coordinate i ?” versus “is coordinate i on?” — and a code can serve the second long after it has stopped serving the first.

Algorithms 1 and 2 state the procedure; Section 4 gives its cost, its applicability conditions and its failure cases, and Table 2 maps each step to the function that implements it.

We are explicit about what is and is not new. The rank-trace mechanism behind the floor is standard in frame theory, and the recovery arguments are standard compressed sensing. The contribution is the packaging

of these facts into a criterion that can be *run* on a network, together with the gain-normalised form that makes it scale-invariant, the $O(Fd^2)$ streaming implementation that makes it feasible at $F \approx 10^6$, and the experimental demonstration that the criterion discriminates between trained models whose interface status is currently under debate.

1.3. Contributions

1. **The Interface Diagnostic.** A computable interface criterion for over-complete codes and trained networks, with pseudocode, hyperparameters, complexity, applicability conditions, edge cases and failure modes (Section 4), and a reference implementation whose every step is traceable to a numbered equation (Table 2). The diagnostic proper (Algorithm 2) is stated for a *fixed* code and readout; the random-code ensemble used for the scaling study (Algorithm 3) is a separate procedure and is not a diagnosis of any code.
2. **A gain-normalised floor, computable in $O(Fd^2)$.** For any G, Φ with $(G\Phi)_{ii} = 1$ the average squared off-diagonal cross-talk is at least $(F - d)/(d(F - 1))$ (Theorem 5.1); Corollary 5.2 removes the normalisation and bounds the *interference-to-gain ratio* $\max_{i \neq j} |M_{ij}|/|M_{ii}|$, making the diagnostic invariant to readout rescaling. Lemma 4.2 then reduces the mean-square statistic to $d \times d$ sufficient statistics, exactly and with no approximation, which is what lets the attainment ratio be evaluated at $F \approx 10^6$; the maximum form admits no such reduction and we state its $O(F^2d)$ cost rather than conflate the two. Proposition 5.5 identifies what a ratio near one means under the pseudoinverse readout — equalised leverage scores, a property of the code’s geometry and not of its usefulness.
3. **The floor of the code itself, and what an attainment ratio actually measures.** Theorem 5.1 constrains an ensemble, so a code sitting a fraction of a percent above it cannot be interpreted: the readout may be excellent, or the floor may be unreachable for that code. Theorem 5.6 removes the ambiguity in closed form — the least cross-talk attainable at unit gain on feature i is $1/h_i - 1$, attained by the calibrated pseudoinverse — and Proposition 5.8 attributes the gap to the code exactly, as a weighted dispersion of its leverage scores. This factors the usual ratio into $R_{\text{geom}} \times R_{\text{readout}}$ (Eq. (5)), and the consequence is deflationary: the calibrated pseudoinverse has $R_{\text{readout}} = 1$ identically, so an attainment

ratio reported with it is a statement about the code’s geometry and says nothing about the decoder. We report both factors.

4. **A per-feature affine capacity, computable exactly.** For a *given* code and a *given* feature, Theorem 7.2 gives the largest sparsity at which any affine probe can detect that feature, as the value κ_i of a single linear programme: separability holds at every sparsity up to s iff $\kappa_i > s$, and the frontier is $\lceil \kappa_i \rceil - 1$. This is the question interpretability practice asks and the ensemble bounds do not answer. Written for supports of size *at most* s , the threshold is s itself, monotonicity holds by construction, and κ_i settles every sparsity at once. Proposition 7.4 shows the state law enters through one scalar only — the smallest detectable amplitude — and Remark 7.6 declines to call that distribution-awareness, because two laws with the same support family give the same frontier. Failure comes with a finite certificate that a third party can re-check from Φ alone (Proposition 7.8), and success comes with a margin equal to half the distance between the two hulls, which is a certified noise tolerance rather than a scale (Proposition 7.7).
5. **The hierarchy is an implication, and only one way.** Sherman–Morrison makes leverage exactly redundancy (Lemma 8.1), which bounds the affine frontier from above (Theorem 8.2) and yields a failure threshold with no decoder, no probe and no training in it: leverage below $\min\{\frac{1}{2}, (s-1)^2/((F-1) + (s-1)^2)\}$ makes affine separability at sparsity s impossible (Corollary 8.4). The converse is false, and $\Phi = [I_d \ I_d]$ refutes it: perfectly uniform leverage, hence optimal analog geometry, and yet no feature separable at any positive sparsity (Proposition 8.5). So bad analog geometry implies a bad affine frontier while good analog geometry implies nothing — which is what makes “hierarchy” literally true here rather than a label on three separate measurements.
6. **A separation between two decoding questions, under one sparse-state model.** Proposition 6.5 gives the average analog-reconstruction floor under uniformly random supports, so that both sides of Corollary 6.6 use the same distribution. At $F = d^2$, thresholding recovers a uniformly random size- s support for $s = O(d/\log d)$ with high probability, while every unit-diagonal rank- d linear readout retains average per-coordinate squared *reconstruction* error $\Omega(s/d)$ on that same distribution. The separation is between reconstructing values and identifying the support; it

is not a claim that nonlinearity is required, and the thresholded linear readout is measured throughout as the appropriate control.

7. **The trained network does not out-decode an affine probe of its own representation.** Across 120 trained models at three widths, with the probe restricted to the same post-ReLU state the network’s output layer reads and both decoders given one validation-selected threshold, the two tie on s_{95} in 119 of 120 (Section 11.4). This outcome was registered before the runs, and it is reported as the finding rather than replaced by a statistic that reverses it. What does separate is the *stage*: the pre-ReLU representation is more affinely decodable than the post-ReLU one in every cell, so the gap earlier work reads as a nonlinear advantage is a statement about what the ReLU discards.
8. **The loss decides the interface, and the geometry is mostly not learned.** An L^2 objective destroys the interface — leverage dispersion drives R_{geom} into the tens and rising with width, while an untrained random code stays at 1.0, and the collision frontier collapses to 1, meaning some feature is not separable even at $s = 1$. An L^4 objective does not. But an untrained random code already reaches $R_{\text{geom}} \approx 1.01$ and comes within one to two sparsity levels of the L^4 frontier, so near-floor geometry is generic at this load and is not evidence of learned structure. The L^2 solution is also shown by direct measurement to maintain an effectively linear interface, an independent and quantitative corroboration of the reading in [3].
9. **Application to computation in superposition.** Matching the floor against the published tolerance of the Hänni correction layer gives a $d^{3/2}$ compatibility scale for that proof template, not a universal upper bound; the Adler–Shavit framework maintains the threshold invariant and is therefore not obstructed by the same criterion.

1.4. Scope of claims

Six caveats apply throughout, and we state them once here rather than repeating them.

First, Theorem 5.1 is a rank–trace, unit-diagonal cross-Gramian inequality closely related to known Welch and dual-frame Welch bounds [5, 6, 7, 8, 9, 10]; its role here is the application to readout interfaces, not a new

frame-theoretic theorem. Second, the threshold-recovery results are standard coherence-based and random-dictionary support-recovery arguments; we claim no novelty in compressed sensing. Third, the $d^{3/2}$ scale we derive for the Hänni template is a compatibility threshold certified by that specific proof pipeline, not a universal upper bound on computation in superposition. Fourth, the experiments cover random codes, gradient-optimised codes and one trained toy model; none of them is a claim about large trained language models.

Fifth, the constrained minimisation behind Theorem 5.6 is the Capon beamformer with the frame operator in place of a covariance, and its minimiser is the canonical dual frame vector rescaled; we claim no novelty in the optimisation, only in reading h_i as leverage and in the exact attribution of Proposition 5.8. Sixth, the margin of Proposition 7.7 and the dimensional ceiling of Proposition 7.10 are stated in the exactly- s parameterisation in which they were derived and verified, while the frontier itself is stated for at most s ; and while the finite nonlinear witness of Section 8.1 is implemented and validated, its compression to $d + 2$ states is not, and no measurement in this paper depends on it.

2. Background and related work

2.1. Computation in superposition

A network of hidden width d represents $F \gg d$ features in superposition if its activation $a(x) \in \mathbb{R}^d$ is approximately Φb for a feature-encoding matrix $\Phi \in \mathbb{R}^{d \times F}$ and a sparse Boolean b . Following [1], features $f_1, \dots, f_F$ are ε -linearly represented by a if there is a readout $R \in \mathbb{R}^{F \times d}$ with $|r_k \cdot a(x) - f_k(x)| < \varepsilon$ for all k and x .

Hänni et al. construct an approximate-linear recursive template for sparse Boolean circuits, alternating a targeted superpositional AND layer with an error-correction layer, and certify emulation of circuits of width $\tilde{O}(d^{3/2})$. Adler and Shavit [2] give parameter-description lower bounds and explicit constructions for computing in superposition, obtaining a near-quadratic capacity by thresholding after each stage. Adler et al. [11] reframe superposition combinatorially through feature channel coding, and Kong et al. [12] show empirically that reducing inter-feature interference at fixed parameter count improves performance. This paper reproves none of these constructions; it identifies the interface-level reason the two regimes coexist.

2.2. Trained toy models of computation in superposition

A recent empirical line studies whether small trained networks actually compute in superposition. Braun et al. [13] introduced the compressed-computation task — a width-50 network asked to compute 100 sparse ReLU features — as a testbed. Bhagat et al. [3] argued that the standard L^2 -trained model largely exploits a mixing term rather than genuinely computing in superposition, and Ferreira da Silva and Heimersheim [4] showed that training under an L^4 loss elicits a solution that does compute in superposition, assigning each feature a sparse codeword over neurons and decoding it with a pseudoinverse of the encoder — that is, a linear readout interface in the sense studied here. In parallel, [14] report perturbation-based evidence for feature-specific error correction in large language models, the mechanism whose certified tolerance drives the $d^{3/2}$ compatibility scale of the Hänni template. This debate is about which decoding interface a trained network maintains; the diagnostic proposed here is a direct instrument for it, and Section 11.3 applies it to exactly the models under dispute.

2.3. Frame theory, Welch bounds and cross-Gramians

Classical Welch bounds [5] lower-bound the correlations of overcomplete unit-norm systems; the maximum-correlation form we use, $\max_{i \neq j} |\langle \phi_i, \phi_j \rangle| \geq \sqrt{(F - d)/(d(F - 1))}$, appears for instance as Theorem 2.3 of Strohmer and Heath [6] in the study of Grassmannian frames. Waldron [7] showed that generalised Welch-bound-equality sequences are exactly tight frames, and Datta, Howard and Cochran [8] derived the entire family of higher-order Welch bounds from a geometric rank/Grammian argument, of which the rank–trace computation behind Theorem 5.1 is the first-order instance; see also [15] for the design-theoretic view. For pairs of sequences, Aceska and Kaczanowski [9] introduced the cross-frame potential, and Christensen, Datta and Kim [10] proved Welch-type bounds for the maximum coherence of dual frame pairs, characterising equality via equiangular tight frames.

Theorem 5.1 sits in this lineage: it is a unit-diagonal cross-Gramian formulation assuming only $\text{rank}(M) \leq d$ and $M_{ii} = 1$ — in particular, no duality relation between the two sequences, which, as finite sequences, are of course frames for their spans. We include its short rank–trace proof only to keep the paper self-contained; the proof mechanism is standard in this literature, cf. [8]. The contribution here is the application to readout interfaces, the gain-normalised form (Corollary 5.2), and the diagnostic built on top of them.

2.4. Compressed sensing and sparse recovery

The threshold-recovery results used below are standard coherence-based and random-dictionary support-recovery arguments. Their role is to make explicit a distinction relevant to neural computation: small-error linear readout requires uniformly small coordinate error, whereas threshold recovery only requires the aggregate interference from the active support to remain below a margin.

The collision frontier of Section 7 sits in this neighbourhood and we place it carefully, because two established literatures ask adjacent questions.

The first is group testing. Recovering a sparse Boolean b from Φb is exactly *quantitative* group testing when Φ is a real measurement matrix, and the combinatorial theory of superimposed, disjunct and separable codes descends from Kautz and Singleton [16]. That theory asks how many tests suffice to identify the *whole* support, by *any* decoder, usually for a matrix to be designed. We ask something weaker and more specific: for a matrix we are given, and one coordinate of it, is that coordinate’s presence decidable by an *affine* function of Φb ? The gap is real in both directions. A matrix can be identifiable in the group-testing sense while $\kappa_i \leq s$, because $\kappa_i \leq s$ only requires a collision between *convex combinations* of admissible states, not an exact coincidence of two integer ones. Conversely $\kappa_i > s$ certifies a linear probe with a margin, which identifiability alone does not supply. The restriction to affine decoders is not a weakness of the analysis but the object of interest, since linear probing is how features are read in practice.

The second is the null space property. Writing z' for z extended by zero at coordinate i , the constraint $\Phi_{-i} z' = \phi_i$ says precisely that $z' - e_i \in \ker \Phi$, so κ_i is a statement about kernel vectors normalised at one coordinate — the same geometry as the null space property and as the neighbourliness of randomly projected polytopes [17]. Two differences matter. The null space property is quantified over all supports and certifies recovery by ℓ_1 minimisation, a nonlinear decoder; κ_i is per-coordinate and certifies an affine one. And the null space property is hard to verify, which is why the literature tests it by semidefinite relaxation [18], whereas κ_i is the exact value of one linear programme — a consequence of asking the weaker, one-coordinate question rather than of a better algorithm. The box $\|z\|_\infty \leq 1/\alpha$, which comes from bounded activations and has no counterpart in the null space property, turns out to bind rarely, so for most features the frontier is a minimum- ℓ_1 -representation quantity and the tools of that literature apply directly.

2.5. Sparse autoencoders

Sparse autoencoder work scales dictionary learning to millions of latents [19, 20, 21] and studies reconstruction–sparsity trade-offs [22]. Such results motivate the importance of overcomplete internal representations, but dictionary size is a reconstructive quantity and does not measure the recursive Boolean interface capacities studied here. This paper answers one of the open questions listed by Sharkey et al. [23] concerning what theoretical insight can be gleaned from computation performed natively in superposition.

3. Problem setting and notation

We write $\tilde{O}(\cdot)$, $\tilde{\Omega}(\cdot)$, $\tilde{\Theta}(\cdot)$ to hide factors polynomial in $\log d$. Table 1 fixes the notation used throughout.

Two decoding questions are compared throughout. *Analog reconstruction* asks that the scores $z = Mb$ approximate b in a norm; *support recovery* asks only that $\mathbf{1}\{z_i \geq \theta\} = b_i$ for every i . The first is a statement about magnitudes and is never exactly satisfiable when $F > d$; the second is a statement about a decision boundary and can be satisfied exactly — by a thresholded linear score as much as by anything else, which is why that rule is measured here too rather than treating support recovery as the preserve of nonlinearity. Everything below is an elaboration of that asymmetry.

4. The Interface Diagnostic

4.1. Problem statement

Problem 4.1 (Interface diagnosis). **Input:** a code $\Phi \in \mathbb{R}^{d \times F}$ with $F > d$ (or a trained network from which an effective code is read off); a readout family $\mathcal{G}$; a sparse-state model, here uniformly random supports of size s drawn from a grid $\mathcal{S}$; a threshold θ ; a trial budget T ; a confidence level $1 - \alpha$.

Output: (i) the attainment ratio $r = \widehat{W}/W(F, d) \geq 1$ and the maximum-form ratio $r_{\max}$, together with its factorisation $R_{\text{geom}} \times R_{\text{readout}}$ against the code’s own floor (Section 5.1); (ii) for each $s \in \mathcal{S}$, the empirical $p_{\text{rec}}(s)$ with a Wilson interval and the exact per-coordinate RMS error $\rho_{\text{lin}}(s)$ of the calibrated linear interface; (iii) the criterion-separation witness $(s_{95}, \rho_{\text{lin}}(s_{95}))$; (iv) for each feature in a chosen subset, the collision frontier κ_i and hence the largest sparsity at which any affine probe can detect that feature (Algorithm 4).

Table 1: Notation.

symbol	meaning
d	activation width (number of neurons)
F	number of features, $F > d$
$\Phi \in \mathbb{R}^{d \times F}$	code; column ϕ_i is the direction of feature i
$G \in \mathbb{R}^{F \times d}$	linear readout
$M = G\Phi \in \mathbb{R}^{F \times F}$	readout interface, $\text{rank}(M) \leq d$
$D = \text{diag}(M_{11}, \dots, M_{FF})$	diagonal gains; $D^{-1}M$ is the calibrated interface
$A = M - I_F$	cross-talk matrix (zero diagonal after calibration)
$b \in \{0, 1\}^F$	sparse Boolean state; $S = \text{supp}(b)$, $s = S $
$\mu = \max_{i \neq j} \langle \phi_i, \phi_j \rangle $	coherence of a unit-norm code
$W(F, d) = \frac{F-d}{d(F-1)}$	mean-square Welch-type floor
$\widehat{W}$	measured mean squared off-diagonal cross-talk
θ	decoding threshold (default 1/2)
$p_{\text{rec}}(s)$	probability of exact threshold recovery at sparsity s
s_{95}	largest s such that $p_{\text{rec}} \geq 0.95$ for <i>every</i> tested $s' \leq s$
$\Sigma = \Phi\Phi^\top$	frame operator (invertible iff Φ has full row rank)
$h_i = \phi_i^\top \Sigma^{-1} \phi_i$	leverage of feature i ; $\sum_i h_i = d$
$W_\star(\Phi)$	the code's own floor: least mean-square cross-talk it admits
$R_{\text{geom}}, R_{\text{readout}}$	$W_\star(\Phi)/W(F, d)$ and $W(G, \Phi)/W_\star(\Phi)$
Φ_{-i}	Φ with column i deleted
$\alpha \in (0, 1]$	smallest detectable active amplitude
$\kappa_i(\alpha)$	collision frontier of feature i ; separable at s iff $\kappa_i > s$
$\delta_i(s), \gamma_i(s)$	distance between the two hulls, and the margin $\delta_i(s)/2$
m, m'	feature counts <i>inside imported statements</i> ; m' plays the role of F

The output is a diagnosis, not a score to be optimised, and the four parts answer different questions. A code with r close to 1 is Welch-optimal in the mean-square sense, but only R_{readout} says anything about the decoder; a large s_{95} with $\rho_{\text{lin}}(s_{95})$ well above $d^{-1/2}$ says the code supports support recovery in a regime where analog reconstruction from the same map is useless; and κ_i is the only part that is per-feature and exact rather than aggregate and empirical, which is what makes it usable as a statement about an individual feature rather than about the code on average.

Algorithm 1 FLOORATTAINMENT — how close the readout interface sits to the floor

Require: code $\Phi \in \mathbb{R}^{d \times F}$, $F > d$; readout $G \in \mathbb{R}^{F \times d}$ or a rule in $\{\text{TIED}, \text{PINV}, \text{LSQ}\}$

Ensure: attainment ratio r (and $r_{\max}$ if requested), or DEGENERATE

- 1: $G \leftarrow \Phi^\top$, Φ^+ , or the least-squares fit, per the rule
 - 2: $D_{ii} \leftarrow \langle g_i, \phi_i \rangle$ for $i = 1, \dots, F$ $\triangleright$ the diagonal of $M = G\Phi$, in $O(Fd)$; M itself is not formed
 - 3: **if** $\min_i |D_{ii}| < \epsilon_{\text{gain}}$ **then return** DEGENERATE $\triangleright$ some feature is read out with vanishing gain
 - 4: **end if**
 - 5: $\bar{G} \leftarrow D^{-1}G$ $\triangleright$ gain normalisation; see Corollary 5.2
 - 6: $H \leftarrow \bar{G}^\top \bar{G}$; $\Sigma \leftarrow \Phi \Phi^\top$ $\triangleright$ both $d \times d$; $O(Fd^2)$
 - 7: $\widehat{W} \leftarrow \frac{\text{tr}(H\Sigma) - F}{F(F-1)}$ $\triangleright$ exact, by Lemma 4.2
 - 8: $W \leftarrow (F-d)/(d(F-1))$ $\triangleright$ Theorem 5.1
 - 9: **if** the maximum form is requested **then**
 - 10: form $\widetilde{M} \leftarrow \bar{G}\Phi$ and set $\widehat{\mu} \leftarrow \max_{i \neq j} |\widetilde{M}_{ij}|$ $\triangleright O(F^2d)$ time, $O(F^2)$ memory
 - 11: **end if**
 - 12: **return** $r \leftarrow \widehat{W}/W$ (and $r_{\max} \leftarrow \widehat{\mu}/\sqrt{W}$ if computed)
-

4.2. Algorithm 1: floor attainment

Step 3 is not a numerical guard but a semantic one: without it, an interface can be made to look arbitrarily good by shrinking G , which is exactly the confusion that Remark 5.3 dispels. By Theorem 5.1, a correct implementation always returns $r \geq 1$; we use that as a runtime assertion, and no violation was observed in 720 gain-normalised interfaces (Section 11.1).

The mean-square statistic that the theorem actually constrains never requires the interface to be built. The maximum statistic does, and we say so rather than folding it into the same cost.

Lemma 4.2 (Mean-square cross-talk from $d \times d$ statistics). *Let $\bar{G} = D^{-1}G$ with $D_{ii} = \langle g_i, \phi_i \rangle \neq 0$, let $\widetilde{M} = \bar{G}\Phi$, and set $H = \bar{G}^\top \bar{G}$ and $\Sigma = \Phi \Phi^\top$, both in $\mathbb{R}^{d \times d}$. Then*

$$\sum_{i \neq j} \widetilde{M}_{ij}^2 = \|\bar{G}\Phi\|_F^2 - F = \text{tr}(H\Sigma) - F.$$

Algorithm 2 SEPARATIONPROFILE — support recovery versus irreducible analog error, on a fixed code

Require: code $\Phi \in \mathbb{R}^{d \times F}$ and readout $G \in \mathbb{R}^{F \times d}$; sparsity grid $\mathcal{S}$; trials T ; threshold θ ; seed σ

Ensure: $\{(s, p_{\text{rec}}(s), \text{CI}, \rho_{\text{lin}}(s))\}_{s \in \mathcal{S}}$ and the separation witness $(s_{95}, \rho_{\text{lin}}(s_{95}))$

- 1: $s_{\max} \leftarrow \max \mathcal{S}$; $\bar{G} \leftarrow D^{-1}G$ as in Algorithm 1
 - 2: $\|A\|_F^2 \leftarrow \text{tr}(H\Sigma) - F$; $\|A\mathbf{1}\|^2 \leftarrow \|\bar{G}\Phi\mathbf{1}\|^2 - 2\mathbf{1}^\top \bar{G}\Phi\mathbf{1} + F \triangleright$ Lemma 4.3;
 $O(Fd^2)$
 - 3: $e_s \leftarrow \frac{1}{F}[(p - q)\|A\|_F^2 + q\|A\mathbf{1}\|^2]$, $p = \frac{s}{F}$, $q = \frac{s(s-1)}{F(F-1)}$, for each $s \in \mathcal{S} \triangleright$
 exact; Prop. 6.5
 - 4: **for** $t = 1$ **to** T **do**
 - 5: draw nested supports $S_1 \subset \dots \subset S_{s_{\max}}$ from seed (σ, t)
 - 6: $Z \leftarrow \bar{G}[x_1, \dots, x_{s_{\max}}]$, $x_s = \sum_{j \in S_s} \phi_j \triangleright O(Fd s_{\max})$
 - 7: $k_s \leftarrow k_s + 1$ for each s with $\mathbf{1}\{Z_{\cdot s} \geq \theta\} = \mathbf{1}_{S_s}$
 - 8: **end for**
 - 9: $p_{\text{rec}}(s) \leftarrow k_s/T$ with Wilson interval; $\rho_{\text{lin}}(s) \leftarrow \sqrt{e_s}$
 - 10: $s_{95} \leftarrow \max\{s : p_{\text{rec}}(s') \geq 0.95 \forall s' \leq s\}$
 - 11: **return** the profile and $(s_{95}, \rho_{\text{lin}}(s_{95}))$
-

Proof. $\widetilde{M}$ has unit diagonal by construction, so the F diagonal entries contribute exactly F to $\|\widetilde{M}\|_F^2$ and the remainder is the off-diagonal sum. Cyclicity of the trace gives $\|\bar{G}\Phi\|_F^2 = \text{tr}(\Phi^\top \bar{G}^\top \bar{G}\Phi) = \text{tr}(\bar{G}^\top \bar{G}\Phi\Phi^\top) = \text{tr}(H\Sigma)$. $\square$

Forming H and Σ costs $O(Fd^2)$ time and $O(d^2)$ memory, against $O(F^2d)$ and $O(F^2)$ for the direct route — a reduction of a factor F/d in time and F^2/d^2 in memory, with no approximation whatever. The identity is verified against the dense computation to relative accuracy 10^{-10} in the accompanying tests.

4.3. Algorithm 2: separation profile of a given code

Algorithm 2 is the diagnostic proper: it takes a code and a readout that already exist — a designed code, an optimised one, or the effective code of a trained network — and holds them fixed while the support varies. This is the only version that says anything about a particular object, and it is the one applied to trained networks in Section 11.3.

Algorithm 3 ENSEMBLESCALING — the same two criteria, averaged over random codes

Require: d, F , sparsity grid $\mathcal{S}$, trials T , threshold θ , seed σ , block size B
Ensure: the ensemble-averaged profile and $(s_{95}, \rho_{\text{lin}}(s_{95}))$

- 1: $s_{\max} \leftarrow \max \mathcal{S}$; $k_s \leftarrow 0$ for all s ; $e_s \leftarrow 0$ for all s
- 2: **for** $t = 1$ **to** T **do**
- 3: draw nested supports $S_1 \subset \dots \subset S_{s_{\max}}$ from seed (σ, t)
- 4: $X \leftarrow$ partial sums $[x_1, \dots, x_{s_{\max}}]$, $x_s = \sum_{j \in S_s} \phi_j$ $\triangleright d \times s_{\max}$
- 5: $\Sigma \leftarrow 0_{d \times d}$, $v \leftarrow 0_d$
- 6: **for** each block β of B columns of Φ **do**
- 7: regenerate Φ_β from seed (σ, t, β) $\triangleright \Phi$ is never stored
- 8: $Z \leftarrow \Phi_\beta^\top X$; mark s as failed if $\mathbf{1}\{Z_{\cdot s} \geq \theta\}$ disagrees with $\mathbf{1}_{S_s}$ on this block
- 9: $\Sigma += \Phi_\beta \Phi_\beta^\top$; $v += \Phi_\beta \mathbf{1}$
- 10: **end for**
- 11: $k_s += 1$ for each surviving s
- 12: $\|A\|_F^2 \leftarrow \|\Sigma\|_F^2 - F$; $\|A\mathbf{1}\|^2 \leftarrow v^\top \Sigma v - 2\|v\|^2 + F$ $\triangleright$ Lemma 4.3
- 13: $e_s += \frac{1}{F} [(p - q) \|A\|_F^2 + q \|A\mathbf{1}\|^2]$, $p = \frac{s}{F}$, $q = \frac{s(s-1)}{F(F-1)}$ $\triangleright$ Prop. 6.5
- 14: **end for**
- 15: $p_{\text{rec}}(s) \leftarrow k_s/T$ with Wilson interval; $\rho_{\text{lin}}(s) \leftarrow \sqrt{e_s/T}$
- 16: $s_{95} \leftarrow \max\{s : p_{\text{rec}}(s') \geq 0.95 \ \forall s' \leq s\}$
- 17: **return** the profile and $(s_{95}, \rho_{\text{lin}}(s_{95}))$

The support-recovery side of Algorithm 2 is Monte Carlo over supports; the analog side is *exact* for the given code, computed once before the loop rather than estimated inside it. Total cost is $O(Fd^2 + T F d s_{\max})$ time and $O(Fd + d^2)$ memory: the $F \times F$ interface is never formed.

4.4. A scaling experiment on the random-code ensemble

Algorithm 3 is a different object and we keep it named apart. It redraws a fresh code at every trial, so what it characterises is the (d, F) random-code *ensemble*, not any individual code; it is the vehicle for the width-scaling study of Sections 11.1 and 11.5, and it is not a diagnosis of anything. Because the code is regenerated block by block from a counted seed and never stored, it reaches widths at which the code would not fit in memory.

Both algorithms share the same asymmetry: the support-recovery side is estimated, the analog side is computed. That is what makes the comparison

a witness rather than two noisy curves — the analog error carries no Monte Carlo uncertainty, so any gap at s_{95} is attributable to the recovery side alone.

Lemma 4.3 (Sufficient statistics of the tied interface). *Let Φ have unit-norm columns, $M = \Phi^\top \Phi$, $A = M - I_F$, $\Sigma = \Phi \Phi^\top \in \mathbb{R}^{d \times d}$ and $v = \Phi \mathbf{1} \in \mathbb{R}^d$. Then*

$$\|A\|_F^2 = \|\Sigma\|_F^2 - F, \quad \|A\mathbf{1}\|_2^2 = v^\top \Sigma v - 2\|v\|_2^2 + F.$$

Proof. By cyclicity of the trace, $\|\Phi^\top \Phi\|_F^2 = \text{tr}((\Phi^\top \Phi)^2) = \text{tr}((\Phi \Phi^\top)^2) = \|\Sigma\|_F^2$; the diagonal of M is all ones and contributes F , so subtracting F leaves the off-diagonal energy, which is $\|A\|_F^2$. For the second identity, $A\mathbf{1} = \Phi^\top \Phi \mathbf{1} - \mathbf{1} = \Phi^\top v - \mathbf{1}$, hence $\|A\mathbf{1}\|_2^2 = v^\top \Phi \Phi^\top v - 2\mathbf{1}^\top \Phi^\top v + F = v^\top \Sigma v - 2\|v\|_2^2 + F$, using $\mathbf{1}^\top \Phi^\top v = v^\top v$. $\square$

Lemma 4.3 is the reason the diagnostic scales: it replaces an $F \times F$ object by a $d \times d$ one, and combined with Proposition 6.5 it yields the average linear error at every sparsity without a single Monte Carlo draw.

4.5. Hyperparameters, complexity and cost

The diagnostic has five hyperparameters: the readout rule, the threshold θ (default $1/2$, the natural midpoint for unit-norm codes and Boolean states), the sparsity grid $\mathcal{S}$, the trial budget T , and the block size B , which trades memory for the number of regeneration passes. Only T affects the statistical resolution; Section 11.1 reports the threshold sensitivity of the conclusions.

Algorithm 1 costs $O(Fd^2)$ time and $O(d^2)$ memory for the ratio r , by Lemma 4.2. The maximum-form ratio $r_{\max}$ is the one exception in the whole method: no $d \times d$ reduction of a maximum is available to us, so obtaining it exactly costs $O(F^2d)$ time and $O(F^2)$ memory, and it is computed only when asked for. Since the floor of Theorem 5.1 is a statement about the mean square, nothing about the floor comparison is lost by omitting it; we report $r_{\max}$ at the moderate widths where it is affordable and drop it beyond. Algorithm 2 costs $O(Fd^2 + T F d s_{\max})$ time and $O(Fd + d^2)$ memory. Algorithm 3 costs $O(T F d s_{\max})$ time — one pass of code regeneration and one $B \times d \times s_{\max}$ product per block — and $O(Bd + d^2)$ memory, independent of F . At $d = 1024$ and $F = 1048576$ the code would occupy 4 GB in single precision; the streaming implementation peaks near $Bd \cdot 4$ bytes. Measured wall-clock times are in Table 8.

4.6. Applicability, edge cases and failure modes

Conditions of applicability. The diagnostic assumes (i) $F > d$, otherwise the floor is vacuous and Algorithm 1 refuses to run; (ii) unit-norm code columns, or a declared normalisation, since the threshold $\theta = 1/2$ is calibrated to unit gain; (iii) a sparse-state model that is exchangeable across coordinates, which is what makes the closed form of step 3 valid.

Edge cases. At $s = 1$ the linear error is still non-zero while threshold recovery succeeds whenever $\mu < \theta$; this is the smallest instance of the separation. When $F/d \rightarrow 1$ the floor tends to zero and the diagnosis becomes uninformative — correctly so, since a nearly complete code has no interference to speak of.

Failure modes. Three are worth naming. (a) *Degenerate gain*: if some $|M_{ii}|$ is numerically zero the calibration is undefined and Algorithm 1 returns DEGENERATE rather than a number; this is not a corner case but the expected outcome when a readout is fitted without the diagonal constraint (Section 11.2). (b) *Uncovered feature*: a least-squares readout fitted on states in which some feature never appears has an identically zero row, which triggers (a); the implementation therefore enforces coverage of the fitting set. (c) *Vacuous profile*: if the coherence of the code exceeds θ , $p_{\text{rec}}(1)$ is already below 0.95 and $s_{95} = 0$; the certificate is then empty, and the diagnostic reports that rather than extrapolating.

4.7. Implementation and traceability

Table 2 maps every quantity in Problem 4.1 to the function that computes it. The correctness of the implementation is checked by a test suite that verifies the theorems numerically — equality for tight frames, no violations for random interfaces, the closed forms against Monte Carlo, and the streaming trial against an explicit dense evaluation of the same code.

5. The linear-readout floor

Theorem 5.1 (Unit-diagonal cross-Gramian Welch floor). *Let $F \geq 2$, let $\Phi \in \mathbb{R}^{d \times F}$ and $G \in \mathbb{R}^{F \times d}$, and set $M = G\Phi$. Suppose $M_{ii} = 1$ for all $i \in [F]$. Then*

$$\sum_{i \neq j} |M_{ij}|^2 \geq \frac{F(F-d)}{d}. \quad (1)$$

Table 2: Each step of the method and the function that implements it. Paths are relative to the repository root.

quantity	where defined	implementation
$W(F, d)$	Eq. (2)	lrtr/codes.py:welch_floor
$D^{-1}M$	Cor. 5.2	lrtr/interface.py:unit_diagonal
$\widehat{W}, \widehat{\mu}, r, r_{\max}$	Alg. 1	lrtr/interface.py:crosstalk_stats
Algorithm 1	Sec. 4	lrtr/diagnostic.py:interface_floor_dia
$\Sigma, v \mapsto \ A\ _F^2, \ A\mathbf{1}\ ^2$	Lem. 4.3	lrtr/diagnostic.py:tied_energy_moments
$\frac{1}{F}\mathbb{E}_S \ A\mathbf{1}_S\ ^2$	Prop. 6.5	lrtr/interface.py:linear_energy_uniform
energy floor	Eq. (6)	lrtr/interface.py:energy_floor_uniform
threshold decoder Q	Thm. 6.1	lrtr/threshold.py:threshold_decode
fixed-code trial	Alg. efalg:two, l. 5–7	lrtr/threshold.py:recovery_trial_fixed
streaming trial	Alg. efalg:ensemble, l. 6–10	lrtr/threshold.py:recovery_trial_strea
Algorithm efalg:two	Sec. efsec:method	lrtr/diagnostic.py:fixed_code_separati
Algorithm efalg:ensemble	Sec. efsec:alg-ensemble	lrtr/diagnostic.py:random_code_scaling
$\text{tr}(H\Sigma) - F$	Lem. eflem:suffstat	lrtr/interface.py:crosstalk_mean_sq
leverage form of r	Prop. efprop:leverage	lrtr/interface.py:crosstalk_mean_sq_pi
s_{95}	Alg. efalg:two, l. 9	lrtr/threshold.py:s95_from_curve

Consequently, if $F > d$,

$$\frac{1}{F(F-1)} \sum_{i \neq j} |M_{ij}|^2 \geq W(F, d) := \frac{F-d}{d(F-1)}, \quad (2)$$

and hence $\max_{i \neq j} |M_{ij}| \geq \sqrt{W(F, d)}$. In particular, if $F/d \rightarrow \infty$ then $\max_{i \neq j} |M_{ij}| \geq (1 - o(1)) d^{-1/2}$.

Proof. Since $M = G\Phi$ with $G \in \mathbb{R}^{F \times d}$ and $\Phi \in \mathbb{R}^{d \times F}$ we have $\text{rank}(M) \leq d$, and the unit diagonal gives $\text{tr}(M) = F$. For any matrix of rank at most d , $|\text{tr}(M)| \leq \|M\|_* \leq \sqrt{d} \|M\|_F$ (Appendix Appendix D), so $F^2 \leq d \|M\|_F^2$. Expanding $\|M\|_F^2 = F + \sum_{i \neq j} |M_{ij}|^2$ gives (1), and dividing by $F(F-1)$ gives (2). For the maximum, note that $\max_{i \neq j} |M_{ij}|^2 \geq \frac{1}{F(F-1)} \sum_{i \neq j} |M_{ij}|^2$, and take square roots. $\square$

The argument is the standard rank–trace mechanism, cf. [8]; we give it in full only for self-containedness. The next corollary is what the diagnostic actually uses, because it removes the normalisation hypothesis.

Corollary 5.2 (Gain-normalised form). *Let $F > d$ and let $M = G\Phi$ satisfy $\text{rank}(M) \leq d$ and $M_{ii} \neq 0$ for all i . Then*

$$\max_{i \neq j} \frac{|M_{ij}|}{|M_{ii}|} \geq \sqrt{\frac{F-d}{d(F-1)}}.$$

Proof. Apply Theorem 5.1 to $D^{-1}M$ with $D = \text{diag}(M_{11}, \dots, M_{FF})$: this matrix has rank at most d , unit diagonal, and entries M_{ij}/M_{ii} . $\square$

Remark 5.3 (Why the gain normalisation is essential). Without a diagonal normalisation the absolute off-diagonal entries of $G\Phi$ can be made arbitrarily small by shrinking G , and the floor appears to fail. A concrete instance: take Φ to be the three unit vectors at 120° in $\mathbb{R}^2$ — an equiangular tight frame — and $G = D\Phi^\top$ with $D = \text{diag}(1/10, 1/20, 1/20)$. Then $G\Phi$ has diagonal $(1/10, 1/20, 1/20)$ and maximum off-diagonal entry $1/20$, ten times *below* $\sqrt{(F-d)/(d(F-1))} = 1/2$. There is no contradiction: this interface reads out its own coordinates with gains far below one, and after recalibrating each row to unit gain the maximum off-diagonal entry becomes exactly $1/2$ — the floor, attained with equality. Corollary 5.2 isolates the scale-invariant content: what is bounded below is the interference-to-gain ratio. This example is verified in the test suite, and Section 11.2 shows that a gradient optimiser left free of the constraint exploits exactly this degree of freedom.

Proposition 5.4 (Equality within the tied family). *Let $\Phi \in \mathbb{R}^{d \times F}$ have unit-norm columns and let $G = \Phi^\top$, so that $M = \Phi^\top \Phi$ has unit diagonal. Then $\sum_{i \neq j} |M_{ij}|^2 \geq F(F-d)/d$, with equality if and only if $\Phi\Phi^\top = (F/d)I_d$, that is, iff Φ is a unit-norm tight frame.*

Proof. The diagonal entries are $\|\phi_i\|^2 = 1$. By cyclicity, $\|M\|_F^2 = \text{tr}((\Phi\Phi^\top)^2) = \|\Sigma\|_F^2$ with $\Sigma = \Phi\Phi^\top \succeq 0$ and $\text{tr } \Sigma = F$. Cauchy–Schwarz on the eigenvalues of Σ gives $\|\Sigma\|_F^2 \geq (\text{tr } \Sigma)^2/d = F^2/d$, with equality iff all d eigenvalues coincide, i.e. $\Sigma = (F/d)I_d$. Subtracting the diagonal contribution F gives the claim. $\square$

Within this family — unit-norm columns read out by their own transpose — equality therefore characterises tight frames, consistent with [7, 8]. We stress the scope: it is a statement about the tied Gram matrix, not about every rank- d unit-diagonal interface. Other readouts attain the same floor under different conditions, and the next proposition identifies the condition for the pseudoinverse. Whether learned codes find such configurations is an empirical question, answered in Section 11.2.

Proposition 5.5 (The pseudoinverse ratio is a leverage statistic). *Let $\Phi \in \mathbb{R}^{d \times F}$ have full row rank, let $G = \Phi^+$, and write $P = \Phi^+ \Phi$ for the orthogonal projector onto the row space, with leverage scores $h_i = P_{ii} \in (0, 1]$. Then the gain-normalised interface $\widetilde{M} = D^{-1}P$ satisfies*

$$\sum_{i \neq j} \widetilde{M}_{ij}^2 = \sum_{i=1}^F \frac{1}{h_i} - F, \quad \text{so} \quad r = \frac{d}{F(F-d)} \left(\sum_{i=1}^F \frac{1}{h_i} - F \right).$$

Since $\sum_i h_i = \text{tr } P = d$, Cauchy–Schwarz gives $\sum_i h_i^{-1} \geq F^2/d$ with equality iff $h_i = d/F$ for every i . Hence $r \geq 1$, and $r = 1$ exactly when leverage is equalised across features.

Proof. P is symmetric idempotent, so row i obeys $\sum_j P_{ij}^2 = P_{ii} = h_i$ and $D_{ii} = h_i$. Dividing row i by h_i and removing the unit diagonal entry leaves $(h_i - h_i^2)/h_i^2 = h_i^{-1} - 1$; summing over i gives the identity. The inequality is Cauchy–Schwarz applied to $\sum_i h_i \cdot \sum_i h_i^{-1} \geq F^2$, and dividing by $F(F-1)$ and by $W(F, d) = (F-d)/(d(F-1))$ turns it into $r \geq 1$. $\square$

This is worth stating because it deflates a natural over-reading of a ratio close to one under the pseudoinverse readout. Attaining the floor with $G = \Phi^+$ says that the code spreads its features evenly over the row space — nothing more. It is a statement about the geometry of the code, not about whether the code is a good representation for any task; a code can have perfectly equalised leverage and still be useless. We use this to scope the empirical claims of Section 11.3.

5.1. The floor of the code itself

Theorem 5.1 bounds every rank- d unit-diagonal interface, so it is a statement about the *ensemble* of codes: no particular Φ need be able to attain it. That makes a measured ratio hard to read. If a trained code sits 0.06% above $W(F, d)$, is the readout excellent, or is the floor simply unreachable for this code and the readout merely adequate? The question is answered by computing the floor of the code itself, which is available in closed form.

Theorem 5.6 (Code-specific analog optimum). *Let $\Phi \in \mathbb{R}^{d \times F}$ have full row rank, let $\Sigma = \Phi \Phi^\top \succ 0$ be its frame operator, and let $h_i = \phi_i^\top \Sigma^{-1} \phi_i$ be the leverage of feature i . Then for every i*

$$\min_{g \in \mathbb{R}^d} \left\{ \sum_{j \neq i} (g^\top \phi_j)^2 : g^\top \phi_i = 1 \right\} = \frac{1}{h_i} - 1, \quad (3)$$

attained uniquely at $g_i^* = \Sigma^{-1}\phi_i/h_i$. Stacking the F minimisers as rows gives $G^* = D_h^{-1}\Phi^+$ with $D_h = \text{diag}(h_1, \dots, h_F)$: the row-wise optimum is the Moore–Penrose pseudoinverse after gain calibration, and coincides with Φ^+ itself only when every $h_i = 1$. Consequently the smallest mean-square off-diagonal cross-talk this code admits is

$$W_*(\Phi) := \frac{1}{F(F-1)} \sum_{i=1}^F \left(\frac{1}{h_i} - 1 \right) = \frac{\sum_i h_i^{-1} - F}{F(F-1)} \geq W(F, d), \quad (4)$$

with equality if and only if $h_i = d/F$ for every i .

Proof. Since $\sum_j (g^\top \phi_j)^2 = g^\top \Sigma g$, the constraint $g^\top \phi_i = 1$ turns the objective into $g^\top \Sigma g - 1$. Minimising a positive-definite quadratic under one linear equality is classical: the Lagrange conditions give $g = \lambda \Sigma^{-1} \phi_i$, the constraint fixes $\lambda = 1/h_i$, and the value is $\phi_i^\top \Sigma^{-1} \phi_i / h_i^2 = 1/h_i$. Subtracting the constrained direction's contribution, which is exactly 1, gives (3); strict convexity gives uniqueness. Summing over i and dividing by $F(F-1)$ gives (4). Finally $\sum_i h_i = \text{tr}(\Sigma^{-1} \Phi \Phi^\top) = d$, so Cauchy–Schwarz gives $\sum_i h_i^{-1} \geq F^2/d$ with equality iff the h_i are all equal, and $(F^2/d - F)/(F(F-1)) = W(F, d)$. $\square$

Remark 5.7 (Provenance). Equation (3) is the minimum-variance distortionless-response (Capon) beamformer with the frame operator in place of a covariance [24], and g_i^* is the canonical dual frame vector rescaled to unit gain; both are classical. We claim no novelty in the optimisation. What the diagnostic uses is the reading of h_i as a leverage score, the resulting per-code floor (4), and the exact attribution of the gap in Proposition 5.8.

Proposition 5.8 (The excess is leverage dispersion, exactly). *With $c = d/F$ the mean leverage,*

$$\sum_{i=1}^F \frac{1}{h_i} - \frac{F^2}{d} = \sum_{i=1}^F \frac{(h_i - c)^2}{c^2 h_i} \geq 0.$$

Hence $W_*(\Phi)$ exceeds $W(F, d)$ by a weighted dispersion of the leverage scores, which vanishes precisely when leverage is uniform.

Proof. Expand the right-hand side: $c^{-2} \sum_i (h_i^2 - 2ch_i + c^2)/h_i = c^{-2} \sum_i h_i - 2c^{-1}F + \sum_i h_i^{-1}$. Substituting $\sum_i h_i = d$ and $c = d/F$ turns the first two terms into $F^2/d - 2F^2/d = -F^2/d$. $\square$

This separates two questions that a single ratio against $W(F, d)$ conflates. Writing $W(G, \Phi)$ for the mean-square off-diagonal cross-talk actually achieved by a readout G ,

$$\underbrace{\frac{W(G, \Phi)}{W(F, d)}}_{\text{what is usually reported}} = \underbrace{\frac{W_\star(\Phi)}{W(F, d)}}_{R_{\text{geom}}: \text{ the code}} \times \underbrace{\frac{W(G, \Phi)}{W_\star(\Phi)}}_{R_{\text{readout}}: \text{ the decoder}}, \quad (5)$$

and the two factors answer different questions. R_{geom} asks whether the code's leverage is spread evenly; $R_{\text{readout}} \geq 1$ asks whether the decoder is the best one *for that code*. In particular Proposition 5.5 says that the calibrated pseudoinverse has $R_{\text{readout}} = 1$ identically, so a published attainment ratio computed with $G = \Phi^+$ measures R_{geom} and nothing else. Section 11.3 reports both factors.

Finally, the analog level reads a state distribution through one object only, which is what lets Section 8 compare it with the affine level under a single state model.

Proposition 5.9 (Distribution-weighted reconstruction error). *Let $M = G\Phi$ have unit diagonal, let $A = M - I_F$, and let the state $b \in \mathbb{R}^F$ be drawn from any law with finite second moment $C = \mathbb{E}[bb^\top]$. Then $\mathbb{E} \|Ab\|_2^2 = \text{tr}(A^\top AC)$. The analog level therefore depends on the state law only through C : two laws with the same second moment impose the same reconstruction error on every readout.*

Proof. $\mathbb{E} \|Ab\|_2^2 = \mathbb{E} \text{tr}(A^\top Abb^\top) = \text{tr}(A^\top AC)$ by linearity. $\square$

Corollary 5.10 (No better-than-floor ε -linear readout). *Let $F > d$ and suppose G satisfies $\|Gy_i - e_i\|_\infty \leq \delta$ for singleton states y_i , with $\delta < 1/2$. Then $\delta/(1 - \delta) \geq \sqrt{(F - d)/(d(F - 1))}$; in particular $\delta = \Omega(d^{-1/2})$ when $F/d \rightarrow \infty$.*

Proof. Let $M = G[y_1, \dots, y_F]$. The hypothesis gives $|M_{ii} - 1| \leq \delta$ and $|M_{ji}| \leq \delta$ for $j \neq i$. Calibrating, $|\widetilde{M}_{ij}| \leq \delta/(1 - \delta)$, and Theorem 5.1 applies to $\widetilde{M}$. $\square$

6. Threshold recovery and the separation

The floor rules out small-error ε -linear readout with $\varepsilon = o(d^{-1/2})$ when $F \gg d$. It does not rule out thresholded Boolean recovery, because thresholding does not require every inactive score to be small — only that the aggregate interference stay below the margin.

Theorem 6.1 (Threshold recovery under coherent superposition). *Let Φ have unit-norm columns with coherence μ , let b be s -sparse, let $x = \Phi b + \eta$ with $\|\Phi^\top \eta\|_\infty \leq \nu$, and let $Q(z)_i = \mathbf{1}\{z_i \geq 1/2\}$. If $s\mu + \nu < 1/2$ then $Q(\Phi^\top x) = b$, with margin $\tau = \frac{1}{2} - (s\mu + \nu) > 0$.*

Proof. For $i \in S = \text{supp}(b)$ we have $z_i = 1 + \sum_{j \in S, j \neq i} \langle \phi_i, \phi_j \rangle + \langle \phi_i, \eta \rangle \geq 1 - (s-1)\mu - \nu \geq \frac{1}{2} + \tau$. For $i \notin S$, $z_i \leq s\mu + \nu = \frac{1}{2} - \tau$. $\square$

Corollary 6.2 (Random codes at quadratic load). *Let $F = \lfloor d^2 \rfloor$ and let $\phi_1, \dots, \phi_F$ be independent uniform unit vectors in $\mathbb{R}^d$. Then $\mu \leq 6\sqrt{\log d/d}$ with probability at least $1 - d^{-5}$ for all sufficiently large d , and on that event the threshold decoder recovers every s -sparse Boolean state whenever $\|\Phi^\top \eta\|_\infty < \frac{1}{2} - 6s\sqrt{\log d/d}$.*

Proof. For independent uniform unit vectors u, v , the inner product $\langle u, v \rangle$ has the law of the first coordinate of a uniform point on S^{d-1} , whose tail satisfies $\Pr\{|\langle u, v \rangle| > t\} \leq 2 \exp(-(d-1)t^2/2)$ for $0 < t < 1$ (see e.g. [25, Thm. 3.4.6]). Taking a union bound over the $\binom{F}{2}$ unordered pairs and using $2\binom{F}{2} \leq F^2 \leq d^4$,

$$\Pr\{\mu > t\} \leq d^4 \exp\left(-\frac{(d-1)t^2}{2}\right).$$

With $t = 6\sqrt{\log d/d}$ we have $\frac{(d-1)t^2}{2} = 18(1 - 1/d) \log d \geq 9 \log d$ for $d \geq 2$, so $\Pr\{\mu > t\} \leq d^4 e^{-9 \log d} = d^{-5}$. The recovery claim is then Theorem 6.1 with $\mu \leq t$. $\square$

The worst-case coherence bound is pessimistic for a *random* support, which is the case the diagnostic measures. The following average-case statement is the one used in Algorithms 2 and 3.

Theorem 6.3 (Random-support threshold recovery). *Let Φ have independent uniform unit-vector columns and let S be a uniformly random support of size s , independent of Φ . For $\delta \in (0, 1)$ there is a universal $C > 0$ such that, with probability at least $1 - \delta$,*

$$\max_{i \in [F]} \left| \sum_{j \in S, j \neq i} \langle \phi_i, \phi_j \rangle \right| \leq C \sqrt{\frac{s \log(F/\delta)}{d}},$$

and on that event thresholding $\Phi^\top(\Phi \mathbf{1}_S + \eta)$ at $1/2$ recovers $\mathbf{1}_S$ whenever $\|\Phi^\top \eta\|_\infty + C\sqrt{s \log(F/\delta)/d} < 1/2$.

Proof. Conditionally on S and on ϕ_i , the summands are independent and each is distributed as the first coordinate of a uniform point on S^{d-1} , hence sub-Gaussian with variance proxy $O(1/d)$ [25, Thm. 3.4.6]. The sum of at most s of them is sub-Gaussian with proxy $O(s/d)$, so $\Pr\{|I_i| > t \mid S\} \leq 2 \exp(-c_0 dt^2/s)$. A union bound over $i \in [F]$ and the stated choice of t give the claim; the recovery statement follows as in Theorem 6.1. Full details are in Appendix C. $\square$

Remark 6.4 (Which randomness the probability is over). The guarantee of Theorem 6.3 is *annealed*: the probability is taken over the joint draw of the code Φ and the support S . It is therefore the right statement for Algorithm 3, which redraws the code at every trial, and it is not directly the statement for Algorithm 2, which holds one code fixed. A quenched version — Φ fixed, probability over S alone — follows from concentration for sampling without replacement, but it is not free: it needs hypotheses on the particular Φ (bounded coherence and bounded row sums of $|\Phi^\top \Phi| - I$), which a given trained code may or may not satisfy. We do not assume them. What the fixed-code diagnostic reports is therefore an empirical recovery rate with a Wilson interval, and the theorem is quoted as the ensemble-level expectation it is, not as a guarantee about the code in hand. The analog side has no such caveat: Proposition 6.5 is exact for every fixed Φ .

We now give the average linear-readout error under the *same* distribution, which is what allows the two criteria to be compared without a distributional mismatch.

Proposition 6.5 (Average linear energy floor, uniform random supports). *Let $F > d$, let $M = G\Phi \in \mathbb{R}^{F \times F}$ satisfy $\text{rank}(M) \leq d$ and $M_{ii} = 1$ for all i , and set $A = M - I_F$. Let $S \subseteq [F]$ be a uniformly random support of size s with $1 \leq s \leq F/2$ and let $b = \mathbf{1}_S$. Then*

$$\mathbb{E}_S \|Ab\|_2^2 \geq \frac{s(F-s)}{F(F-1)} \|A\|_F^2 \geq \frac{s(F-s)(F-d)}{d(F-1)}. \quad (6)$$

In particular $\frac{1}{F} \mathbb{E}_S \|Ab\|_2^2 \geq \frac{s(F-d)}{2d(F-1)}$, which is $\Omega(s/d)$ whenever $F/d \rightarrow \infty$.

Proof. For a uniformly random size- s support, $\mathbb{E}[b_i] = s/F$ and $\mathbb{E}[b_i b_j] = \frac{s(s-1)}{F(F-1)}$ for $i \neq j$. Hence $\mathbb{E}[bb^\top] = (p-q)I_F + q\mathbf{1}\mathbf{1}^\top$ with $p = s/F$ and $q = \frac{s(s-1)}{F(F-1)}$, and

$$p-q = \frac{s(F-1) - s(s-1)}{F(F-1)} = \frac{s(F-s)}{F(F-1)}.$$

Therefore $\mathbb{E}_S \|Ab\|_2^2 = \text{tr}(A^\top A \mathbb{E}[bb^\top]) = (p - q) \|A\|_F^2 + q \|A\mathbf{1}\|_2^2 \geq (p - q) \|A\|_F^2$. Since A has zero diagonal and $\text{rank}(M) \leq d$ with unit diagonal, Theorem 5.1 gives $\|A\|_F^2 \geq F(F - d)/d$, which yields the second inequality. For $s \leq F/2$ we have $F - s \geq F/2$, and dividing by F gives the per-coordinate form. $\square$

Corollary 6.6 (Separation at quadratic load, one sparse-state model). *Let $F = d^2$ and let Φ have independent uniform unit-vector columns. There is a universal $c > 0$ such that, for $1 \leq s \leq cd/\log d$ and a uniformly random support S of size s , thresholding $\Phi^\top \Phi \mathbf{1}_S$ at $1/2$ recovers $\mathbf{1}_S$ exactly with probability at least $1 - d^{-10}$ for all sufficiently large d . For the same sparse-state model, every unit-diagonal rank- d linear readout has average per-coordinate squared error at least $\frac{s(F-d)}{2d(F-1)} = \Omega(s/d)$, and hence RMS error $\Omega(\sqrt{s/d})$.*

Proof. Apply Theorem 6.3 with $\delta = d^{-10}$, so $\log(F/\delta) = 12 \log d$; the interference bound is $C\sqrt{12s \log d/d}$, which is below $1/2$ as soon as $s \leq cd/\log d$ with $c < 1/(48C^2)$. The second statement is Proposition 6.5 with $F = d^2$. $\square$

Remark 6.7 (What the separation compares). The two sides compare different *interface criteria* on the same input distribution: expected squared linear readout error versus exact-recovery probability of a threshold decoder. Neither criterion dominates the other in general. The content of Corollary 6.6 is that at quadratic feature load one of them is attainable exactly and the other is provably not, in the same regime — which is precisely the quantity Algorithm 2 reports. Corollary 6.6 is stated for uniform supports; the Bernoulli variant, with the same conclusion, is in Appendix C.

7. The affine level: a per-feature collision frontier

Sections 5 and 6 bound two criteria on random ensembles. Neither answers the question an interpretability practitioner actually asks about a *particular* trained code: for this feature, in this network, up to what sparsity can a linear probe read the feature’s presence off the representation at all? This section gives that quantity exactly, as the value of one linear programme per feature.

Definition 7.1 (Affine separability of a feature). Fix $\Phi \in \mathbb{R}^{d \times F}$, a feature i and a sparsity s . Write $\mathcal{A}_i(s)$ for the set of states Φb with $b \in \{0, 1\}^F$, $b_i = 1$ and $|\text{supp}(b)| \leq s$, and $\mathcal{B}_i(s)$ for the same with $b_i = 0$. Feature i is *affinely*

separable at sparsity s if there exist $w \in \mathbb{R}^d$ and $\theta \in \mathbb{R}$ with $w^\top x > \theta$ for every $x \in \mathcal{A}_i(s)$ and $w^\top x < \theta$ for every $x \in \mathcal{B}_i(s)$.

The states are Boolean here, and that is not a simplification we could drop. Allowing an active amplitude to be any positive number makes “ i is active” an open condition, and an active state with $b_i \rightarrow 0$ converges to an inactive one; the two hulls then touch for every code and every sparsity, so nothing would ever be separable. A positive floor on the active amplitude is what makes the question well posed, and Corollary 7.5 is the graded statement with that floor made explicit.

An affine functional separates two sets exactly when it separates their convex hulls, so the question is whether those two hulls meet. The programme below decides it.

Theorem 7.2 (The collision frontier). *For $z \in \mathbb{R}^{F-1}$, indexed by the features other than i , let $z_j^+ = \max(z_j, 0)$ and $z_j^- = \max(-z_j, 0)$, so that both parts are non-negative and $z = z^+ - z^-$. Let Φ_{-i} be Φ with column i deleted. Put*

$$\kappa_i := \min_z \left\{ \max \left(\sum_j z_j^+, 1 + \sum_j z_j^- \right) : \Phi_{-i} z = \phi_i, \|z\|_\infty \leq 1 \right\}. \quad (7)$$

with $\kappa_i := +\infty$ when the constraints admit no z at all. Then feature i is affinely separable at every sparsity $s' \leq s$ if and only if $\kappa_i > s$. When κ_i is finite the largest sparsity at which feature i is separable is $\lceil \kappa_i \rceil - 1$; when $\kappa_i = +\infty$ the feature is separable at every sparsity, and no collision exists to bound.

Proof. Separability fails exactly when $\text{conv } \mathcal{A}_i(s)$ meets $\text{conv } \mathcal{B}_i(s)$. By Proposition 7.4 below, applied to the coordinates other than i with budget $s - 1$ on the active side and s on the inactive side, those hulls are $\Phi\{e_i + u : u \in [0, 1]^{F-1}, \mathbf{1}^\top u \leq s - 1\}$ and $\Phi\{v : v \in [0, 1]^{F-1}, \mathbf{1}^\top v \leq s\}$ respectively — the relaxation from 0/1 coordinates to the box is exact, because the polytope is integral. So a collision is a pair u, v in those two boxes with $\Phi(e_i + u) = \Phi v$. We may take u and v to have disjoint supports, since a common component cancels from both sides and only relaxes the two sum constraints. Then $\Phi(e_i + u) = \Phi v$ reads $\Phi_{-i}(v - u) = \phi_i$, so with $z = v - u$ we have $z^+ = v$ and $z^- = u$. The budget on the inactive state is $\sum_j v_j \leq s$, that on the active state is $1 + \sum_j u_j \leq s$, and the entries lie in $[0, 1]$, giving $\|z\|_\infty \leq 1$. The two budgets are exactly $\sum_j z_j^+ \leq s$ and $1 + \sum_j z_j^- \leq s$, so a collision at some sparsity at most s exists if and only if the objective in (7) can be brought to s or below, that is, if and only if $\kappa_i \leq s$. $\square$

Algorithm 4 COLLISIONFRONTIER — the largest affinely separable sparsity, per feature

Require: code $\Phi \in \mathbb{R}^{d \times F}$; feature set $\mathcal{I} \subseteq [F]$; minimum amplitude $\alpha \in (0, 1]$

Ensure: $\kappa_i(\alpha)$ and the frontier $\lceil \kappa_i(\alpha) \rceil - 1$ for each $i \in \mathcal{I}$

1: **for** $i \in \mathcal{I}$ **do**

2: solve, in variables $u, v \in \mathbb{R}_{\geq 0}^{F-1}$ and $t \in \mathbb{R}$:

$\min t \quad \text{s.t.} \quad \Phi_{-i}(u - v) = \phi_i, \quad \alpha \mathbf{1}^\top u \leq t, \quad 1 + \alpha \mathbf{1}^\top v \leq t, \quad u, v \leq 1/\alpha$

3: $\kappa_i(\alpha) \leftarrow t^*$; $s_i^{\max} \leftarrow \lceil \kappa_i(\alpha) \rceil - 1$

4: **end for**

5: **return** $\{(\kappa_i(\alpha), s_i^{\max})\}_{i \in \mathcal{I}}$

Three things improve on the corresponding exactly- s statement, which we record because the earlier formulation of this work used it. The threshold is s itself rather than $2 \min(s, F - s) - 1$; the admissible sets nest in s , so monotonicity of the frontier holds by construction rather than needing a proof and a restriction to $s \leq F/2$; and $\kappa_i = 7.4$ now reads directly as “separable up to 7, lost at 8”, which is what a capacity ought to mean. Computationally (7) is a single linear programme, stated as Algorithm 4: the max of two linear forms is handled by one auxiliary scalar, and one solve settles every sparsity at once rather than one feasibility problem per s .

Remark 7.3 (Why splitting z into two non-negative vectors loses nothing). Algorithm 4 does not constrain u and v to have disjoint supports, so a feasible point need not have $u = z^+$ and $v = z^-$. It is still exact. If $u_j, v_j > 0$ for some j , subtracting $\min(u_j, v_j)$ from both leaves $u - v$ unchanged, keeps feasibility, and strictly decreases both $\mathbf{1}^\top u$ and $\mathbf{1}^\top v$, hence does not increase t . Every optimum therefore admits a canonical split, on which the two budget constraints are exactly those of (7) and the bounds $u, v \leq 1/\alpha$ are exactly $\|z\|_\infty \leq 1/\alpha$.

The cost is one linear programme in $2(F - 1) + 1$ variables with d equality constraints per feature, which is affordable for every feature at $d = 50$ and not at $d = 400$; how we choose a subset there, and why the resulting minimum is an upper bound, is stated in Section 12.1.

7.1. What the state distribution contributes

Definition 7.1 allowed graded activations in $[0, 1]$. It is natural to ask what happens under a genuine state law, and the answer is unusually clean: almost nothing survives.

Proposition 7.4 (Amplitudes wash out of the hulls). *For $\alpha \in (0, 1]$ and $k \in \mathbb{N}$ let $\mathcal{U}_k^\alpha = \{u \in \mathbb{R}_{\geq 0}^F : u_j \in \{0\} \cup [\alpha, 1], |\text{supp}(u)| \leq k\}$. Then $\text{conv} \mathcal{U}_k^\alpha = \{u \in [0, 1]^F : \mathbf{1}^\top u \leq k\}$ for every α .*

Proof. The right-hand side is the unit-weight knapsack polytope, which is integral: its vertices are the 0/1 vectors with at most k ones. Each such vertex lies in $\mathcal{U}_k^\alpha$ for every $\alpha \leq 1$, so the right-hand side is contained in the hull; the reverse inclusion is immediate since $\mathcal{U}_k^\alpha$ satisfies both constraints. $\square$

Corollary 7.5 (The frontier under a minimum amplitude). *If active amplitudes lie in $[\alpha, 1]$ then feature i is separable at every sparsity $s' \leq s$ iff $\kappa_i(\alpha) > s$, where*

$$\kappa_i(\alpha) = \min_z \left\{ \max \left(\alpha \sum_j z_j^+, 1 + \alpha \sum_j z_j^- \right) : \Phi_{-i} z = \phi_i, \|z\|_\infty \leq 1/\alpha \right\},$$

and $\kappa_i(\alpha)$ is non-decreasing in α . As $\alpha \rightarrow 0$ an active state converges to an inactive one, the hulls touch, and no uniform margin exists.

Remark 7.6 (The affine level is a support-level worst case, not a weighted average). It would be convenient to call Corollary 7.5 a distribution-aware frontier. It is not, and the distinction matters. By Proposition 7.4 two state laws with the same support family and the same minimum amplitude give the *same* $\kappa_i(\alpha)$, however differently they weight the states in between. The affine level therefore reads exactly one scalar off the state law — the smallest detectable amplitude — whereas by Proposition 5.9 the analog level reads the full second moment C . That asymmetry is a property of the two criteria, not an artefact of our formulation: separability is a statement about a worst case over a support family, and a worst case cannot be reweighted. A genuinely probabilistic version, guaranteeing separability for all but a δ fraction of states, is possible but needs machinery we do not open here.

7.2. Margin, certificate, and a dimensional ceiling

The verdict of Theorem 7.2 is binary. Two further quantities say how comfortably it holds and how it fails, and one says that no code can do well

beyond a certain sparsity. The first two are stated for the exactly- s parameterisation in which they were derived and verified, where the admissible set carries the affine constraint $\mathbf{1}^\top z = 1$ and an ℓ_1 budget; we flag that rather than restate them without proof.

Proposition 7.7 (The robust margin is half the hull distance). *Let $\delta_i(s) = \text{dist}(\mathcal{A}_i(s), \mathcal{B}_i(s))$ be the Euclidean distance between the two hulls. The maximum-margin affine decoder for feature i has $\|w^*\| = 1/\delta_i(s)$ and margin $\gamma_i(s) = 1/(2\|w^*\|) = \delta_i(s)/2$. Consequently, with the optimal unit-norm w and its centred bias, every perturbed input $x + \eta$ with $\|\eta\|_2 < \gamma_i(s)$ still receives the correct label on every admissible support.*

Proof. The margin programme is $\min \|w\|$ subject to $\min_{u \in D} w^\top u \geq 1$ with $D = \mathcal{A}_i(s) - \mathcal{B}_i(s)$. Support-function duality gives $\max_{\|w\| \leq 1} \min_{u \in D} w^\top u = \text{dist}(0, D) = \delta_i(s)$, whence $\|w^*\| = 1/\delta_i(s)$. For the perturbation claim, the score moves by at most $\|w\| \|\eta\|$. $\square$

The margin is therefore a certified noise tolerance in the units of the representation, not merely a scale, and $\delta_i(s) = 0$ holds exactly when the feature is not separable — the verdict and the margin are two readings of one object.

Proposition 7.8 (Failure comes with a finite, independently checkable certificate). *If feature i is not separable at sparsity s , there are states $x_1^+, \dots, x_m^+ \in \mathcal{A}_i(s)$, states $x_1^-, \dots, x_n^- \in \mathcal{B}_i(s)$ and convex weights λ, μ with*

$$\sum_k \lambda_k x_k^+ = \sum_l \mu_l x_l^-. \quad (8)$$

No affine rule can label all of them correctly, since affinity would force the same point to lie strictly on both sides of θ . By Radon’s theorem the obstruction can be compressed to at most $d + 2$ states.

Remark 7.9 (What the certificate is, and what it is not). Equation (8) is a convex combination of active states equal to a convex combination of inactive ones. It is *not* a single pair of colliding supports, and describing it as one would be wrong: the obstruction is generically spread over several states on each side. This matters practically, because the certificate is what makes the negative verdict auditable. “The solver reported infeasible” is not evidence; a list of supports from which any third party can rebuild the states from Φ

and recompute (8) is. Our implementation stores the supports rather than the cut vectors for exactly that reason, and re-verifies them in a process that never sees the solver.

Proposition 7.10 (A dimensional ceiling on the affine level). *Call a set C of columns an affine circuit if it is minimally affinely dependent: $\sum_{j \in C} l_j \phi_j = 0$ with $\sum_{j \in C} l_j = 0$ and every $l_j \neq 0$. Let q be the smallest size of an affine circuit of Φ . Since any $d + 2$ points in $\mathbb{R}^d$ are affinely dependent, $q \leq d + 2$ whenever $F > d + 1$. Moreover the exactly- s collision radius satisfies $\rho_{\min} \leq q - 1 \leq d + 1$, and hence no code supports featurewise affine support recovery beyond sparsity $\lceil d/2 \rceil$.*

Proof. Given a circuit C , pick $i \in C$ with $|l_i|$ largest and set $z_j = -l_j/l_i$ for $j \in C \setminus \{i\}$ and $z_j = 0$ otherwise. Then $\Phi_{-i}z = \phi_i$; the affine constraint $\mathbf{1}^\top z = 1$ holds automatically, because $\sum_{j \in C} l_j = 0$ forces $\sum_{j \neq i} l_j = -l_i$; $\|z\|_\infty \leq 1$ by the choice of i ; and $\|z\|_1 \leq q - 1$. Hence $\rho_i \leq q - 1$. The sparsity bound follows from the threshold $2 \min(s, F - s) - 1$ of the exactly- s rule. $\square$

This is the affine counterpart of Theorem 5.1: one bounds error from below, the other bounds sparsity from above, and both follow from the ambient dimension alone.

8. The hierarchy is an implication, and only one way

We now have three criteria on the same code: analog reconstruction (Section 5.1), affine support recovery (Section 7), and whatever a trained nonlinear decoder achieves. Calling them a hierarchy is only justified if one implies another. It does, in one direction, and the implication runs through leverage — the same statistic that Theorem 5.6 showed governs the analog level.

Lemma 8.1 (Leverage is redundancy, exactly). *If Φ has full row rank and $h_i < 1$ then $\min\{\|z\|_2^2 : \Phi_{-i}z = \phi_i\} = h_i/(1 - h_i)$.*

Proof. The minimum-norm solution gives $\min\|z\|_2^2 = \phi_i^\top (\Phi_{-i} \Phi_{-i}^\top)^{-1} \phi_i$, and $\Phi_{-i} \Phi_{-i}^\top = \Sigma - \phi_i \phi_i^\top$. Sherman–Morrison gives $(\Sigma - \phi_i \phi_i^\top)^{-1} = \Sigma^{-1} + \Sigma^{-1} \phi_i \phi_i^\top \Sigma^{-1} / (1 - h_i)$, so the value is $h_i + h_i^2 / (1 - h_i) = h_i / (1 - h_i)$. $\square$

So a feature with low leverage is reproducible by the others with small coefficients, with no detour through coherence. That is the missing step: it

converts a statement about the analog geometry into a bound on the affine frontier.

Theorem 8.2 (Low leverage forces the frontier down). *For every i with $h_i \leq \frac{1}{2}$,*

$$\kappa_i \leq 1 + \sqrt{\frac{(F-1)h_i}{1-h_i}}. \quad (9)$$

Proof. Take the z of Lemma 8.1, so $\Phi_{-i}z = \phi_i$ and $\|z\|_2 = \sqrt{h_i/(1-h_i)}$. Then $\|z\|_1 \leq \sqrt{F-1} \|z\|_2$ and $\max(\sum_j z_j^+, 1 + \sum_j z_j^-) \leq 1 + \|z\|_1$. For z to be *admissible* it must also satisfy the box, and $h_i \leq 1/2$ gives $\|z\|_2 \leq 1$ and hence $\|z\|_\infty \leq 1$. So this z witnesses the right-hand side. $\square$

Remark 8.3 (The hypothesis is necessary, not cosmetic). Above $h_i = 1/2$ the minimum- ℓ_2 representation need not satisfy the box, and the inequality is then not merely unproved but false. Take unit-norm columns in $\mathbb{R}^2$,

$$\phi_1 = (1, 0), \quad \phi_2 = \left(\frac{1}{4}, \frac{\sqrt{15}}{4}\right), \quad \phi_3 = \left(\frac{1}{4}, -\frac{\sqrt{15}}{4}\right).$$

Here $h_1 = 8/9$, so the right-hand side of (9) equals 5. But the second coordinate forces $z_2 = z_3$ and the first then forces $z_2 = z_3 = 2$, so the only representation of ϕ_1 by the other two columns violates $\|z\|_\infty \leq 1$: the programme (7) is infeasible and $\kappa_1 = +\infty$. An earlier version of this work stated (9) for all $h_i < 1$; that was wrong, and the counterexample is in the test suite.

Corollary 8.4 (A failure threshold with no decoder in it). *If*

$$h_i \leq \min \left\{ \frac{1}{2}, \frac{(s-1)^2}{(F-1) + (s-1)^2} \right\}$$

then feature i is not affinely separable at sparsity s .

Proof. Solve $1 + \sqrt{(F-1)h/(1-h)} \leq s$ for h , intersect with the hypothesis of Theorem 8.2, and apply Theorem 7.2. $\square$

The cap only binds for $s-1 > \sqrt{F-1}$; below that the second expression is already at most a half and the statement is the unrestricted one. Dropping it makes the corollary false, and by the same mechanism as Remark 8.3: with $a = (1, 0)$, $b = (-\frac{25}{44}, \frac{\sqrt{1311}}{44})$ and $\phi_i = (-\frac{17}{40}, \frac{\sqrt{1311}}{40})$, all unit-norm in $\mathbb{R}^2$, the

unique representation is $z = (\frac{1}{5}, \frac{11}{10})$, so $\kappa_i = +\infty$ and the feature is separable at every sparsity — yet $h_i = 5/9$ lies below the uncapped $s = 3$ threshold of $2/3$.

Corollary 8.4 is a statement about the code alone: no training, no decoder, no probe. It predicts affine collapse from a single leverage score. Its cost is looseness — the $\sqrt{F-1}$ step is lossy, and Section 11.3 reports a case where it predicts failure at $s = 11$ against a measured frontier of 4. It is a sufficient condition for collapse, not an estimate of the frontier.

Proposition 8.5 (The converse is false). *Let $\Phi = [I_d \ I_d] \in \mathbb{R}^{d \times 2d}$. Then $h_i = 1/2$ for every i , so leverage is perfectly uniform and $W_\star(\Phi) = W(F, d)$: the analog geometry is optimal. Yet $\kappa_i = 1$ for every i , the smallest value the frontier can take, so no feature is separable at any positive sparsity.*

Proof. $\Sigma = 2I_d$, so $h_i = \phi_i^\top \phi_i / 2 = 1/2$ and Theorem 5.6 gives equality. Each column i has a duplicate j with $\phi_j = \phi_i$; taking $z = e_j$ gives $\Phi_{-i}z = \phi_i$ with $\sum_j z_j^+ = 1$, $\sum_j z_j^- = 0$ and $\|z\|_\infty = 1$, so $\kappa_i \leq \max(1, 1) = 1$. The reverse holds for every code and every feature, since $z^- \geq 0$ forces the objective to be at least $1 + \sum_j z_j^- \geq 1$. Hence $\kappa_i = 1$. Theorem 7.2 requires $\kappa_i > s$, and $1 > 1$ is false, so feature i is separable at *no* positive sparsity: the largest separable sparsity is $\lceil \kappa_i \rceil - 1 = 0$. Directly: the singleton state e_i and the singleton state e_j have identical representations, so no rule of any kind can tell them apart. $\square$

Together, Theorem 8.2 and Proposition 8.5 say:

Bad analog geometry *implies* a bad affine frontier. Good analog geometry does *not* imply a good one.

That is a strict one-way hierarchy, with the forward arrow proved and the converse refuted by an explicit code, and it is what makes the word “hierarchy” literally true here rather than a label attached to three separate measurements.

8.1. The nonlinear level: a witness, not a frontier

The third level has no frontier to compute. “All nonlinear decoders” has no capacity without a restricted class, and restricting it leads to sign-rank, which we do not open. What it does have is a finite certificate, and it comes free from Proposition 7.8: the states appearing in (8) are explicit vectors that

no affine rule can label correctly, and Radon’s theorem says the obstruction compresses to at most $d + 2$ of them. A trained network either resolves them or does not, with margin

$$\gamma_{\text{net}} = \min \left(\min_k N_i(x_k^+) - \theta, \min_l \theta - N_i(x_l^-) \right),$$

which is a sharper report than a recovery curve: not “a solver says the hulls intersect somewhere among the $\binom{F}{s}$ admissible states” but a named, listed set that any third party can rebuild from Φ and check.

The extraction is implemented (`lrtr.affine_frontier.radon_witness`) and validated by asking a linear programme for an affine separator of exactly the extracted states and requiring it to be infeasible; on small codes it is also checked against brute-force enumeration of every Boolean state. Two honest qualifications. The compression to $d + 2$ is *not* implemented, so the witness we return is valid but not minimal — on the codes we have examined it comes out at about $d + 2$ states already, which is small against $\binom{F}{s}$ but is not a handful, and the per-state margin γ_{net} is therefore a minimum over that many states. And no result in this paper is yet derived from it; the chain $G_1 \rightarrow G_2 \rightarrow G_3$ is complete as a construction, and reporting witnesses for trained models is future work rather than a claim made here.

9. Application: two frameworks, two invariants

We now apply the interface distinction to the two frameworks whose apparent tension motivated this work. All statements imported from the source papers were checked against arXiv:2408.05451v1 and arXiv:2409.15318v3; the audit is recorded in Appendix [Appendix B](#).

Lemma 9.1 (Computation-layer interface; restated from [1, Thm. 11]). *Fix $s = O(1)$ and suppose the targeted superpositional AND primitive of Hänni et al. is applied in a regime satisfying its stated graph-balancing, sparsity, norm and incoming-interference hypotheses. Then the computation layer produces outgoing readout error $\varepsilon_{\text{comp}}(d) = \tilde{O}(\sqrt{s^2/d}) = \tilde{O}(d^{-1/2})$.*

Lemma 9.2 (Correction-layer tolerance; restated from [1, Thm. 21]). *The correction layer admits incoming error $\varepsilon_{\text{in}} < K(d) d^{1/4} / (F^{1/2} s^{1/4})$ with $K(d) = \text{polylog}(d)$, and produces $\varepsilon_{\text{out}} = O((\log d) \sqrt{s/d})$ and $\mu_{\text{out}} = \tilde{O}(\sqrt{s/d})$.*

Proposition 9.3 (Template compatibility threshold; derived from Lemmas 9.1–9.2). *Fix $s = O(1)$. The published Hänni computation–correction template is certified in the regime $F \lesssim \tilde{O}(d^{3/2})$, and its recursive construction emulates circuits of width m using width $d = \tilde{O}(m^{2/3}s^2)$, giving a matching certified scale $\tilde{\Theta}(d^{3/2})$. This is a template-specific certificate, not an impossibility result for other implementations.*

Proof. The correction theorem applies after the computation layer when $\varepsilon_{\text{comp}}(d) \lesssim \varepsilon_{\text{corr,max}}(F, d, s)$, i.e. $\tilde{O}(d^{-1/2}) \lesssim K(d) d^{1/4}/F^{1/2}$ for $s = O(1)$. Suppressing polylogarithmic factors, $F^{1/2} \lesssim d^{3/4}$, hence $F \lesssim \tilde{O}(d^{3/2})$. Conversely, inverting $d = \tilde{O}(m^{2/3}s^2)$ at $s = O(1)$ gives $m = \tilde{\Omega}(d^{3/2})$. $\square$

In the Adler–Shavit framework, exact Boolean state is restored by thresholding after every stage. Their parameter-description lower bound $\Omega(m' \log m')$, converted to neurons under their architectural assumption of $\Theta(n^2)$ parameters with $O(1)$ average description length each, gives $F_{\text{rec}}^{\text{AS}}(n) \leq O(n^2/\log n)$; their explicit construction uses $n = O(\sqrt{m'} \log m')$ neurons, giving $F_{\text{rec}}^{\text{AS}}(n) \geq \Omega(n^2/\log^2 n)$. The remaining logarithmic gap is open.

The two regimes do not contradict each other, and the reason is now precise. The Hänni template maintains an approximate ε -linear interface, so it must feed its residual error into a correction theorem whose tolerance is $\tilde{O}(d^{1/4}/F^{1/2})$; matching that tolerance to the unavoidable $d^{-1/2}$ cross-talk scale of Theorem 5.1 gives the $d^{3/2}$ threshold. Adler–Shavit instead threshold, leaving the small-error class entirely, so Corollary 5.10 does not apply to them. In the vocabulary of Section 4: the first framework operates where the diagnostic’s attainment ratio matters, the second where its criterion-separation witness does.

10. Experimental setup

Six campaigns validate the diagnostic, from controlled random codes to a trained network. E1–E3 establish that the implementation measures what the theory describes; E4 tests whether *learned* codes attain the floor; E5 applies the diagnostic to trained models; E6 tests how the recovery threshold scales.

Environment.. All experiments run on CPU only, with no CUDA device. Python 3.12.10, NumPy 2.4.4, SciPy 1.17.1, PyTorch 2.13.0+cpu on a 14-core machine; thread counts are pinned and two cores are reserved for the

system. Every campaign writes a run record with the exact command, seeds, configuration, environment, wall-clock duration and exit status. Total measured compute is 343.8 minutes (Table 8).

E1 — floor on random codes.. $d \in \{16, 32, 64, 128\}$, $F/d \in \{2, 4, 8, 16\}$, 15 independent codes per configuration, three readouts (tied, pseudoinverse and an i.i.d. Gaussian readout as an adversarial control), and a harmonic tight frame at each (d, F) as the equality witness.

E2 — recovery and separation at $F = d^2$.. $d \in \{64, 128, 256\}$, 200 trials per configuration, thresholds $\theta \in \{0.4, 0.5, 0.6\}$ (3 values) as a sensitivity ablation.

E3 — average linear energy.. $d \in \{32, 64, 128\}$ at $F = 8d$, sparsities $s \in \{1, 2, 4, 8, 16, 32, 64, 128\}$, energies computed in closed form under both the Bernoulli and the uniform-support model, with a 20 000-draw Monte Carlo cross-check of the closed form.

E4 — optimised codes.. $d \in \{16, 32, 64\}$, $F/d \in \{2, 4, 8, 16\}$ (12 configurations), Adam at learning rate 10^{-2} for 20 000 steps, 168 runs in total across four variants: *free* (G and Φ independent), *tied* ($G = \Phi^\top$), *smooth-max*, and the *uncalibrated* ablation in which the diagonal constraint is removed.

E5 — trained toy model.. The compressed-computation task of [13]: $d = 50$, $F = 100$, inputs with each coordinate active with probability $p \in \{0.01, 0.02\}$ and value $\sim U[-1, 1]$, target $\text{ReLU}(x)$, $\hat{y} = W_{\text{out}}\text{ReLU}(W_{\text{in}}x)$ trained by Adam for 50 000 steps under an L^2 and an L^4 loss, 5 seeds each, plus an i.i.d. random-code control at the same (d, F) . Diagnosis uses 400 evaluation trials per sparsity and three linear readouts, including a least-squares readout fitted on the evaluation distribution — deliberately the strongest linear baseline we can give the competing criterion.

E6 — scaling.. $d \in \{128, 256, 512, 1024\}$ at $F = d^2$, up to $F = 1\,048\,576$ features. The code is never materialised: each trial regenerates it in blocks from a counted seed sequence and evaluates every sparsity in one pass using nested supports.

Statistics.. Recovery probabilities carry Wilson 95% intervals. Group comparisons in E5 use a two-sided exact Mann–Whitney U test with Cliff’s δ as effect size. The scaling law in E6 is a one-parameter least-squares fit of $s_{95} = cd/\ln d$, the shape being predicted by the theory and only the constant free; with 4 points the fit is reported as a consistency check, not as an estimate with inferential force.

Reproducibility.. Every figure and table in this paper is generated from the raw result files by a script, and every measured number in the text is a macro emitted from those same files; a checker regenerates the macros, compares them against the file on disk, and fails the build on any mismatch, on a macro the text uses but does not define, and on a macro defined but never used. We are explicit about the limit of that guarantee: it certifies that the *values* match the saved results, not that the sentences around them draw the right conclusion. Prose claims still need a reader. Commands are in Appendix [Appendix A](#).

A note on E6’s linear side.. To keep the cost of the largest widths manageable, the exact linear-energy statistic is computed on every fifth trial rather than every trial, so its averages rest on fewer samples than the recovery curves (10 trials against 50 at $d = 1024$). The recovery probabilities use the full budget.

11. Results

11.1. E1–E3: the diagnostic measures what the theory describes

Across 16 configurations, 15 trials each and three readouts — 720 calibrated interfaces in total — the floor was violated 0 times. This is a correctness check on the implementation rather than evidence for Theorem [5.1](#), and we report it as such.

The attainment ratio of the tied readout falls from 1.999 at the lowest feature load to 1.063 at the highest (Table [3](#), Fig. [1a](#)): a random code is close to Welch-optimal in the mean-square sense once F/d is large, since $\mathbb{E}\langle\phi_i, \phi_j\rangle^2 = 1/d$ while $W(F, d) \rightarrow 1/d$ from below. At $d = 128$, $F = 2048$ the tied ratio is 1.0662 and the pseudoinverse ratio is 1.0010, the latter being the closer of the two at high load. In all 16 configurations the harmonic tight frame attains the floor to within 4e-16 of exact equality, confirming Proposition [5.4](#) at machine precision. The Gaussian control, by contrast,

reaches ratios as large as $3.4\text{e}+08$: the bound holds for it too, but nothing forces an arbitrary readout to be anywhere near the floor, and the diagnostic’s attainment ratio is therefore informative across seven orders of magnitude rather than a formality.

Fig. 1b shows threshold recovery at $F = d^2$. The transition moves right with d , from $s_{95} = 1$ at $d = 64$ to $s_{95} = 2$ at $d = 256$. At $d = 256$ and $s = s_{95}$, thresholding is exact in at least 95% of trials while the calibrated linear interface retains a per-coordinate RMS error of 0.0884, which is 1.41 times the $d^{-1/2}$ scale and above its own theoretical floor of 0.0882. That pair of numbers is the criterion-separation witness of Problem 4.1, measured.

The threshold is a hyperparameter, and $\theta = 1/2$ is not a privileged choice. At $d = 256$ the recovery threshold moves from $s_{95} = 1$ at $\theta = 0.4$ to $s_{95} = 2$ at $\theta = 0.5$ and $s_{95} = 4$ at $\theta = 0.6$. Raising it helps here because, with unit-norm codes and Boolean states, active scores concentrate near one while inactive scores concentrate near zero, so a higher cut trades false positives for false negatives favourably in this regime; the individual interference terms are of course signed. Two limits on how far this ablation goes: the three thresholds re-decode the *same* trials rather than independent replications, so they show the sensitivity of the decision rule and not of the sampling; and at the smallest width the effect is not merely numerical — at $d = 64$, $\theta = 0.4$ no sparsity reaches the 95% level at all, so no certificate exists there.

E3 confirms the energy scale (Fig. 1c). Over 24 configurations the ratio $E/(s/d)$ lies in $[0.9735, 1.0088]$ — the s/d reference scale is essentially exact for random codes — with 0 violations of either energy floor, and the closed forms agree with a 20 000-draw Monte Carlo estimate to within a relative 0.0013.

Measured against the *proved* uniform-support floor of Proposition 6.5, however, the ratio lies in $[1.1395, 2.2165]$: the bound is correct but conservative by a factor between one and two in this regime. That is expected and worth stating plainly, since it bounds what the diagnostic can certify. Two steps of the proof give the slack: the non-negative $q \|A\mathbf{1}\|^2$ term is discarded, and $\|A\|_F^2$ is replaced by its worst case $F(F - d)/d$ rather than the value realised by the code at hand. The looser the feature load, the larger the second effect — here $F = 8d$, so $(F - d)/F = 7/8$ alone accounts for part of the gap. Algorithm 2 therefore reports the *exact* energy computed from the code’s own frame operator, and uses the floor only as the guarantee that the quantity can never be zero.

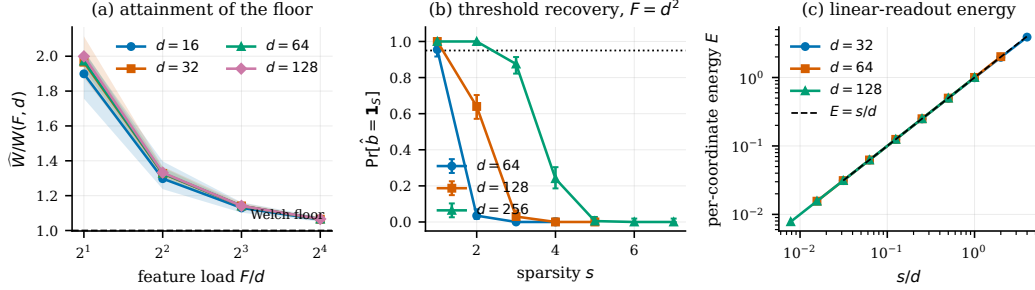


Figure 1: **The Interface Diagnostic on random codes.** (a) Attainment ratio $\widehat{W}/W(F, d)$ of the calibrated tied readout versus feature load; shaded bands are the min-max range over trials, the dashed line is the floor of Theorem 5.1. (b) Exact threshold recovery at $F = d^2$ with Wilson 95% intervals; the dotted line marks the 0.95 level defining s_{95} . (c) Average per-coordinate linear energy under uniformly random supports against the s/d reference. Generated by `scripts/make_figures.py` from `results/e1`, `results/e2` and `results/e3`.

Table 3: E1: attainment ratio $\widehat{W}/W(F, d)$ of the calibrated tied readout for random unit-norm codes, averaged over 15 trials. Values above one are required by Theorem 5.1; the approach to one with growing feature load is the content of the measurement.

d	F/d			
	2	4	8	16
16	1.899	1.298	1.130	1.063
32	1.966	1.327	1.141	1.065
64	1.972	1.329	1.141	1.066
128	1.999	1.332	1.141	1.066

11.2. E4: learned codes attain the floor

Gradient descent under the unit-diagonal constraint drives the interface to the floor (Table 4, Fig. 2). Over 60 runs of the free variant, spanning all 12 configurations, the attainment ratio reaches a mean of 1.0006 and never exceeds 1.0176, with 0 runs below the floor. The tied variant is sharper still: over 36 runs, mean ratio 1.0000 with a maximum of 1.0000 (0 below the floor), and a relative tight-frame residual of 0.0013 on average (worst case 0.0016). In other words, the optimiser recovers the equality case of Proposition 5.4: a unit-norm tight frame emerges from gradient descent rather than being constructed.

The smooth-max variant is the one that does *not* reach its target, and

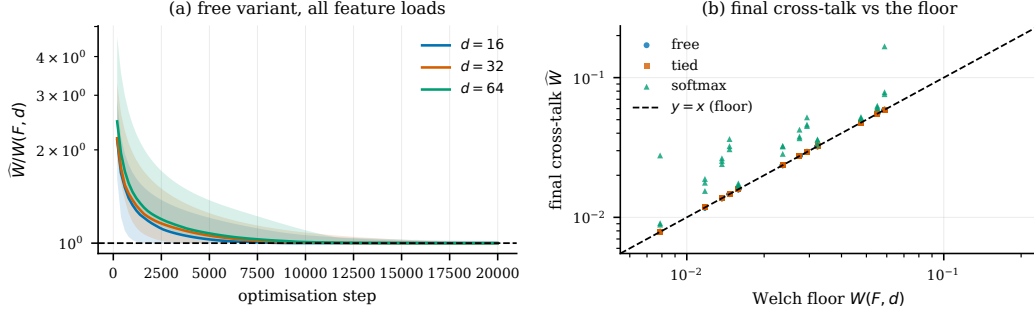


Figure 2: **Optimised codes attain the Welch floor.** (a) Attainment ratio during optimisation of the free variant; bands are min-max over configurations and seeds. (b) Final cross-talk against the floor across every configuration and variant; the dashed line is equality. Generated by `scripts/make_figures.py` from `results/e4`.

the reason is structural rather than a tuning failure. Over 36 runs it attains a mean mean-square ratio of 1.5050 (worst case 3.5230, 0 below the floor) and drives the maximum off-diagonal entry only to 3.125 times $\sqrt{W(F, d)}$. Equality in the maximum form requires an equiangular tight frame, and those exist only for special (d, F) — none of our 12 configurations is guaranteed to admit one. The maximum form of the bound is therefore approached but not attained here, which is the expected outcome and not evidence against the bound.

The uncalibrated ablation is the most informative of the four, and its outcome is more extreme than we expected. Removing the diagonal constraint does not lead the optimiser to a code with low interference: it leads it to the trivial solution. Across all 36 runs the *raw* off-diagonal energy collapses to 6.61e-09 — the readout is driven to zero — and with it the diagonal gains, whose smallest absolute value reaches 0 in single precision. In 25 of the 36 runs the calibration is therefore undefined and Algorithm 1 returns DEGENERATE; in the remaining runs the calibrated interface still respects the floor, and in no run did it fall below (0 violations). An unconstrained optimiser reports spectacular apparent cross-talk while destroying the interface it was supposed to build. This is Remark 5.3 as an experiment, and it is the reason step 4 of Algorithm 1 exists and reports a status rather than a number.

11.3. E5: the separation inside a trained network

Table 5 and Fig. 3 report the diagnostic applied to the compressed-computation models. Three findings stand out; one of them we did not

Table 4: E4: attainment of the floor by gradient-optimised codes, 168 runs over 12 configurations. “Below floor” counts runs whose calibrated interface fell under the theoretical bound; it must be zero. The uncalibrated ablation has no meaningful calibrated ratio by construction.

variant	runs	mean $\widehat{W}/W$	max $\widehat{W}/W$	below floor	
free (G, Φ independent)	60	1.0006	1.0176	0	
tied ($G = \Phi^\top$)	36	1.0000	1.0000	0	
smooth-max	36	1.5050	3.5230	0	
uncalibrated (ablation)	36	n/a	n/a	0	raw ratio $\geq 1.5e - 16$, min $ M $

anticipate, and one prediction we had made was not borne out.

The L^4 -trained network attains the floor.. Its calibrated linear interface has attainment ratio 1.0006 under the pseudoinverse readout (range $[1.0005, 1.0006]$ over 5 seeds) and 1.0216 under the fitted least-squares readout (range $[1.0206, 1.0223]$), against a floor of $W = 0.01010$. The L^2 baseline sits at 28.8135 and the random-code control at 1.0550. E4 had shown the floor to be reachable by direct optimisation of the cross-talk; here it is approached by a network optimising a task loss that never mentions interference.

Two caveats on the statistics, both cutting against reading too much into them. First, the comparison with the random code is readout-dependent: L^4 is closer to the floor than an i.i.d. code of the same shape under the pseudoinverse (1.0006 versus 1.0170) and under least squares, but its own trained decoder is *farther* from the floor (1.0260) than either. That last figure has no counterpart in the control and we do not pair it with one: the random code has no trained decoder, so its W_{out} column is by construction the pseudoinverse and would be compared against a different object. Across the readouts that do admit a matched comparison, the trained code is comparable to a random one on this axis, not better. Second, every pairwise test returns $p = 0.0079$ with $|\delta| = 1$, which is exactly the smallest two-sided p an exact Mann–Whitney can produce at $n = 5$ per group: within-group variance is negligible, so the test reports “no overlap” and nothing more. The effect sizes are what separate the cases, and they differ by orders of magnitude — 28.8135 versus 1.0216 for L^2 against L^4 , a few percent for L^4 against random. No multiplicity correction is applied across the nine tests, and at this n none would survive one.

The separation is real but much narrower than we predicted, and the s_{95} statistic runs against us.. Our pre-registered expectation was that the network’s thresholded output would remain exact over a strictly wider sparsity range than its best linear readout. The measurement contradicts that at the sparsities where both work. Ranking the three linear readouts by exact-recovery rate at each s and taking the best of them, the network is behind up to $s = 4$: at $s = 3$ both are exact, and at $s = 4$ the network recovers in 0.844 of trials while the calibrated W_{out} interface recovers in 0.985. On the s_{95} statistic the linear readout is strictly ahead – $s_{95} = 4$ against the network’s 3 on all 5 seeds at $p = 0.010$, and ahead on 3 of 5 seeds (tied on the remainder) at $p = 0.020$ (Table 5). The preregistered comparison is not supported.

The network only leads further out, where both decoders are already unreliable: at $s = 5$, 0.745 versus 0.294, and at $s = 6$, 0.426 versus 0.136. E5 therefore shows the network extending the usable sparsity range past the point where the thresholded linear readouts of its own code have collapsed, not dominating them everywhere.

The floor statement is on firmer ground, but it is worth being exact about what it is. At every sparsity analog reconstruction carries irreducible error: at $s = 3$, where the network is exact in 1.000 of trials, the best linear readout still has per-coordinate RMS error 0.1733 against its floor of 0.1714. That non-zero error is Theorem 5.1 restated for this code, not an independent measurement — any rank- d unit-diagonal interface has it by construction. What E5 measures is its *size* for a code that gradient descent actually produced, and how far the thresholded decoder gets before that error stops it.

The L^2 model is maintaining a linear interface.. This is the finding we did not plan for. In the L^2 models the ReLU is active on only 4.6% of preactivations, the learned decoder sits at relative distance 0.011 from the pseudoinverse of the encoder, and the network’s thresholded decision agrees with that of a plain linear readout on 99.1% of evaluation states. The L^4 models are elsewhere: ReLU active on 48.6% of preactivations, decoder at relative distance 0.726, and agreement with the linear readout down to 21.8%. The diagnostic thus reproduces, from interface measurements alone and without any mechanistic reverse-engineering, the distinction that [3] and [4] reached by other means. It also explains why the L^2 model is bound by Theorem 5.1 at all: a network that keeps an effectively linear readout *is* a linear readout for these inputs, and its measured ratio of 28.8135 says how far from optimal

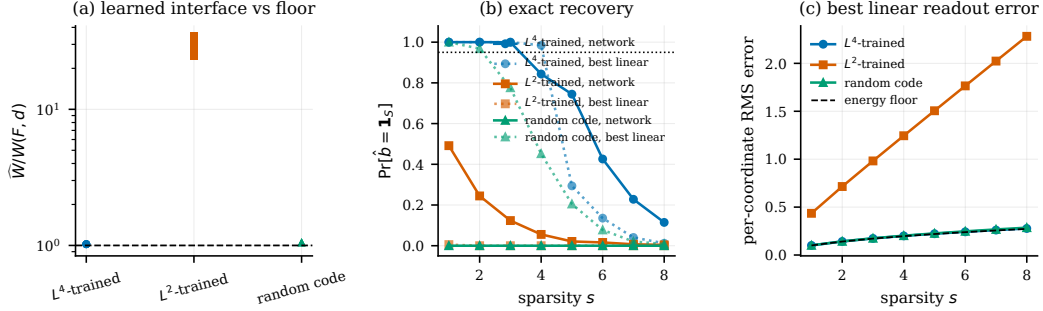


Figure 3: **Interface separation inside a trained toy model of computation in superposition.** A width-50 network trained to compute 100 sparse ReLU features (setup of [13]; L^4 variant of [4]); one point per seed. (a) Attainment ratio of the learned calibrated interface against the floor of Theorem 5.1. (b) Exact recovery by the network’s thresholded output (solid) versus the best linear readout of the same code (dotted). (c) Per-coordinate RMS error of the best linear readout against the energy floor of Proposition 6.5. Generated by `scripts/make_figures.py` from `results/e5`.

that readout is. Its thresholded output recovers even a single active feature only 0.491 of the time.

The finding replicates at a second training sparsity.. Everything above is measured at $p = 0.010$. Repeating the whole campaign at $p = 0.020$ (ablation A5, 5 fresh seeds per condition) reproduces both halves: the L^4 models sit at 1.0217 against the floor and agree with a linear readout on only 22.3% of states, while the L^2 models sit at 45.1973 and agree 99.7% of the time. The interface distinction is therefore not an artefact of one input sparsity.

What does not favour our hypothesis.. Three things. The L^4 models have a *worse* mean squared error on the training task than the L^2 models (1.19e-03 versus 8.30e-04), as expected since they optimise a different objective: the interface advantage is not a task-performance advantage, and we do not claim it is. The s_{95} prediction above failed. And the random-code control is a reminder that a good interface is not by itself evidence of computation: its *linear* readout recovers exactly with probability 0.698 at $s = 3$, better than the L^2 network, simply because an i.i.d. code is already near-Welch-optimal (ratio 1.0550). What the random control lacks is the nonlinear stage; the “network” column for that row is not meaningful and is reported only for completeness.

Table 5: E5: the diagnostic applied to trained models, averaged over 5 seeds at training sparsity $p = 0.010$. “Agrees with linear” is the fraction of evaluation states on which the network’s thresholded decision coincides with that of a plain linear readout — the quantity that identifies which interface the model maintains.

model	$\widehat{W}/W$	s_{95} (network)	s_{95} (best linear)	ReLU clipped (%)	agrees with linear (%)
L^4 -trained	1.022	3.0	4.0	48.6	21.8
L^2 -trained	28.814	0.0	0.0	4.6	99.1
random code	1.055	0.0	2.0	50.0	13.8

What the attainment ratio was measuring

The ratios above are referenced to $W(F, d)$, and Section 5.1 showed that such a ratio conflates two questions. Applying the factorisation (5) to the same models settles which of the two the reported numbers answered, and the answer is uncomfortable: one of them, entirely.

Under the calibrated pseudoinverse the readout factor is 1 to six decimals in every cell — 1.000000 for L^4 , 1.000000 for L^2 , 1.000000 for the random control — which is not a measurement but Theorem 5.6 restated, since the calibrated pseudoinverse *is* the code-specific optimum. The whole ratio is therefore R_{geom} : 1.0006 against a reported 1.0006 for L^4 , 19.0052 against 19.0052 for L^2 , 1.0170 against 1.0170 for the random code. So the claim that the L^4 model comes “within a few parts in a thousand of the floor” is a statement about the evenness of its leverage profile, and contains no information about its decoder. We restate it that way.

Asking the decoder question separately reverses the ordering. Measured against the best readout of its *own* code, the L^4 network’s trained output layer sits at $R_{\text{readout}} = 1.0254$ while the L^2 network’s sits at 1.0053 — so on this criterion the L^2 model has the better decoder and the worse code, the opposite of the reading the single ratio invites. The random control’s own readout gives exactly 1.0000, which is a consistency check rather than a finding: that baseline sets $W_{\text{out}} = \Phi^+$ by construction, so its “own decoder” is the code-specific optimum by definition, and any other value would indicate a bug.

The mechanism is visible in the leverage profiles, and it is the quantity Corollary 8.4 needs. The L^4 code spreads leverage almost uniformly, from 0.4801 to 0.5192 with coefficient of variation 0.0169; the random code is comparable at 0.3927 to 0.6263 (coefficient of variation 0.0918); the L^2 code

is not, running from 0.0092 to 0.9942 with coefficient of variation 0.8673. That dispersion is exactly the excess of Proposition 5.8, and it grows with the training density: at $p = 0.02$ the L^2 geometry factor rises to 29.1032.

Two limits on what this licenses. The factorisation is arithmetic on Theorem 5.6 and needs no new assumption, but it reinterprets an existing measurement rather than adding evidence — five seeds at one width remain five seeds at one width. And 0.0092 is the number Corollary 8.4 consumes: with $s = 2$ the threshold $\min\{\frac{1}{2}, (s-1)^2/((F-1) + (s-1)^2)\}$ is exactly $1/F$, so at $F = 100$ it is 0.01, and the measured minimum leverage of the L^2 code falls below it. The theory therefore predicts that this code loses affine separability at $s = 2$ — from a single leverage score, with no decoder and no probe in the argument. The corollary is applied at the minimum leverage, which is where it is strongest and, being below a half, also where its hypothesis holds. Whether that prediction survives at other widths is the question Section 12.1 flags as open.

11.4. E7: the network against an affine probe of its own representation

The experiments above compare the network’s thresholded output against *fixed* readouts of its code. That understates the baseline, and the audit that prompted this campaign said so. This section replaces it: 180 models at $d \in \{50, 100, 200\}$ with $F = 2d$, 20 seeds per cell, three code families (L^4 , L^2 , and an untrained random code), and an affine probe fitted over three families, two representations, three threshold policies and a six-point regularisation grid, all selected on validation with the test split opened once. `tests/test_probes.py` poisons the test set and asserts that no selected hyperparameter moves, so leakage is excluded structurally rather than by inspection.

Splits, and where they cannot be disjoint.. Train, validation and test draw *disjoint supports*, which is a stronger guarantee than disjoint states and is what makes a fitted probe’s test score meaningful. It cannot hold unconditionally: at $s = 1$ there are only F supports in total — 100, 200 and 400 at the three widths — against the thousands of states each split requests. The sampler therefore serves the test split first, caps the smallest sparsities at what the space contains, and distributes the remainder proportionally; it records which sparsities were exhausted and how many draws were rejected, and it raises rather than silently returning an empty split. At $F = 100$ the $s = 1$ test set holds 50 states, not 4096, and the sparsities $s \in \{1, 2\}$ are

Table 6: E7: network minus best post-ReLU affine probe, s_{95} , paired within seed with a bootstrap interval over the 20 models of each cell. Both decoders read the same state and each gets one validation-selected global threshold.

loss	d	mean difference	95% interval	ties / probe wins
L^4	50	0.00	[0.00, 0.00]	20 / 0
L^4	100	0.00	[0.00, 0.00]	20 / 0
L^4	200	-0.05	[-0.15, 0.00]	19 / 1
L^2	50	0.00	[0.00, 0.00]	20 / 0
L^2	100	0.00	[0.00, 0.00]	20 / 0
L^2	200	0.00	[0.00, 0.00]	20 / 0

flagged exhausted. Every per-sparsity rate at those sparsities is therefore computed on fewer states than the nominal budget, and its Wilson interval is correspondingly wider.

The comparison has to be matched, and matching it changes the answer.. A first pass compared the network at a fixed $\theta = 1/2$ against the best of all eighteen probe configurations. That stacks three advantages on the probe: two thirds of the configurations tune their threshold on validation while the network does not; two thirds read the *pre*-ReLU state Φb while the network’s output layer reads $\text{ReLU}(\Phi b)$; and $\theta = 1/2$ is not on a common scale across a ridge prediction, a logistic probability and a hinge margin. Removing all three — probe restricted to the post-ReLU state, one validation-selected global threshold for each side, differences paired within seed — gives Table 6.

The two tie in 119 of the 120 trained models. This is the outcome registered before the campaign ran, in the words it was registered in: a properly trained, validation-selected affine decoder *matches* the network, and we report that rather than looking for a statistic that reverses it.

The second estimand disagrees, and its sign depends on the width.. The analysis plan named two co-primary estimands: s_{95} and the recovery AUC, the area under the exact-recovery curve over the evaluated sparsities. s_{95} is a floored crossing point and therefore blunt, so a difference of less than one level is invisible to it; the AUC sees the whole curve. On the same matched post-ReLU comparison, the AUC does not tie, and it does not point one way either. At $d = 50$ the network leads by 0.0101 (95% interval [0.0093, 0.0109]) on 20 of 20 seeds; at $d = 100$ the probe leads by -0.0164 ([-0.0170, -0.0158])

on 20 of 20; at $d = 200$ the probe leads by -0.0198 ([-0.0207, -0.0190]) on 20 of 20. Every cell is unanimous across seeds, no interval covers zero, and the direction reverses between the first width and the other two.

We report this because we registered it, and we decline to build on it. The honest reading is that the network holds a small edge away from the 95% crossing at the smallest width and loses it as the width grows, which is the opposite of a nonlinear advantage that strengthens with scale. Whether the reversal reflects the ReLU truncating a growing fraction of the state or the probe’s threshold having more room to help at larger F , three widths cannot say. Figure 4(a,b) shows both estimands so that a reader can see the disagreement rather than take our word for which one to believe.

What does separate is the stage, not the decoder.. Restoring the pre-ReLU probe recovers a gap, and it is a gap between representations rather than between decoders: at $d = 50$ the pre-ReLU probe leads the network by -1.00 of a sparsity level on all 20 seeds, and at $d = 200$ by -1.00. The preactivation is more affinely decodable than the state the ReLU leaves behind. Read as a decoder comparison this looks like a nonlinear disadvantage; read correctly it says what the nonlinearity discards, and the network is not entitled to the information it destroys.

The loss decides the interface.. The three code families separate cleanly and by a wide margin, at every width:

	$d = 50$	$d = 100$	$d = 200$
R_{geom}, L^4	1.0006	1.0003	1.0002
R_{geom}, L^2	18.5635	24.9782	29.8734
$R_{\text{geom}}, \text{random}$	1.0187	1.0099	1.0049
κ_{min}, L^4	4.4873	6.0155	8.0352
κ_{min}, L^2	1.0000	1.0016	1.0146
$\kappa_{\text{min}}, \text{random}$	3.1612	4.4289	6.2377

L^2 drives R_{geom} into the tens and rising with width, and its frontier collapses to 1 — by Theorem 7.2 some feature is not separable even at $s = 1$, which for singleton states means two features share a representation. The mechanism is a spread of column norms: a feature whose direction is a fraction of a typical length lies inside the convex hull of the others, and no duplicate is needed. L^4 does not do this, and its R_{readout} of 1.0263 says its own output layer is near-optimal for the code it built.

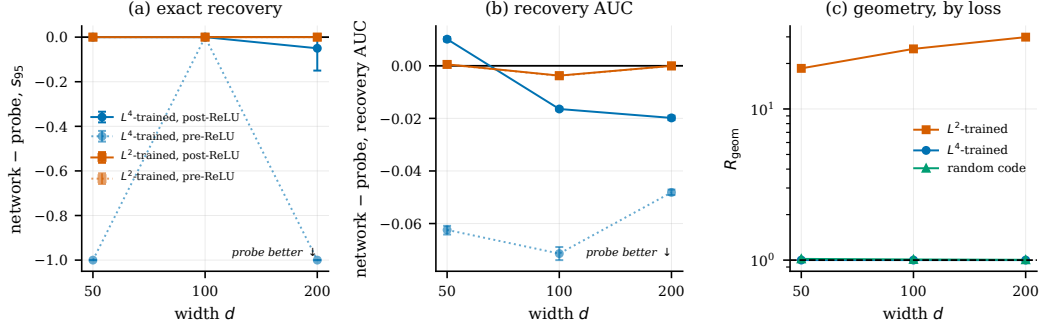


Figure 4: **The matched comparison, both estimands, and the geometry the loss produces.** Paired within seed; bars are bootstrap intervals over the 20 models of a cell. Negative means the affine probe decodes better. **(a)** Under s_{95} the matched post-ReLU comparison is a tie at every width, while the pre-ReLU probe — reading the state the network’s output layer never sees — wins a full level at $d = 50$ and $d = 200$ and ties at $d = 100$. **(b)** The same models under the recovery AUC, which resolves differences smaller than one sparsity level: the sign reverses between $d = 50$, where the network leads on every seed, and the two larger widths, where the probe does. **(c)** R_{geom} by loss. L^2 leaves the interface one to two orders of magnitude above the rank-trace floor; L^4 and an *untrained* random code both sit on it, which is why proximity to the floor is not by itself evidence of learned structure.

But near-floor geometry is generic.. The untrained random code reaches $R_{\text{geom}} = 1.0187$ at $d = 50$ and its frontier is 6.2377 at $d = 200$, against L^4 ’s 8.0352. So the distance from the rank-trace floor, which earlier work reads as a signature of learned structure, is what an i.i.d. code gives for free at this load; the trained code is better, by one to two sparsity levels, and that margin is the honest size of the effect.

And the margin comes from the code, not the readout.. The random control differs from a trained model in two ways at once — its code is not trained *and* its readout is Φ^+ rather than learned — so it cannot say which half matters. E8 separates them with three arms on an identical budget, 10 seeds each: both trained; a random code held *frozen* with only W_{out} trainable; and neither trained.

arm	$d = 50$			$d = 100$			$d = 200$		
	R_{geom}	κ_{min}	R_{ro}	R_{geom}	κ_{min}	R_{ro}	R_{geom}	κ_{min}	R_{ro}
both trained	1.0006	4.468	1.0258	1.0003	6.014	1.0228	1.0002	8.060	1.0185
frozen code	1.0427	2.880	1.8978	1.0209	4.122	1.9217	1.0097	5.874	1.9201
neither	1.0201	3.106	1.0000	1.0100	4.373	1.0000	1.0048	6.272	1.0000

R_{ro} abbreviates R_{readout} . The frozen arm is at $d \in \{50, 100\}$ from one run and $d = 200$ from a second, added after review asked for the third width; the two carry separate run records rather than being merged.

Training the readout alone leaves the frontier where an untrained code puts it, at all three widths: 4.122 against 4.373 at $d = 100$ and 5.874 against 6.272 at $d = 200$, while training both reaches 6.014 and 8.060. The gain is entirely in Φ . That is unsurprising for κ , which is a function of the code alone — the frozen and untrained rows must agree up to the column normalisation that distinguishes them, and their agreement is a consistency check rather than a result — but it is the point: whatever the training buys for support recovery, it buys by moving the code.

The readout column is the surprise, and it is stable across width. The frozen arm trained W_{out} for the full budget and ended at $R_{\text{readout}} = 1.9217$ at $d = 100$ and 1.9201 at $d = 200$ — nearly twice the cross-talk of the best readout *of the code it was given* — while the jointly trained arm reaches 1.0228 and 1.0185.

It is tempting to read that as gradient descent failing at the conditional problem, and it is not. The cross-talk optimum is attained by Φ^+ for *every* code, so a readout with R_{readout} near 1 always exists and its existence settles nothing; what matters is whether the arm’s own objective prefers its W_{out} to that optimum. It does, decisively. Scored under the loss the arm trained on, in its own post-ReLU representation, on fresh draws from a seed no training used (`scripts/frozen_readout_tradeoff.py`), the frozen W_{out} beats Φ^+ by a factor of 4.70 at $d = 100$ and 4.50 at $d = 200$, on 10 of 10 models at each width. The cross-talk it gives up is bought, not lost. As a check on the scoring, the untrained arm — whose readout *is* Φ^+ — returns a ratio of 1.00. Figure 5(b) places both arms in the two coordinates at once, which is the only view in which the trade-off is visible as a trade-off.

So the two objectives genuinely pull apart on a fixed random code, and the contrast is about what co-adaptation does to them rather than about

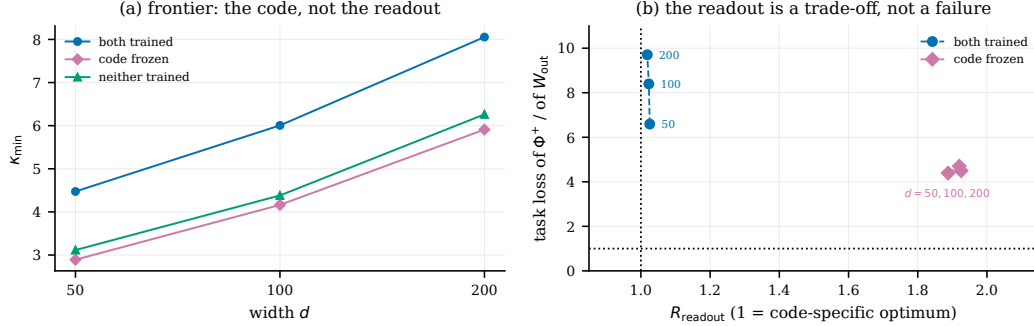


Figure 5: **Which half of the training does the work.** Medians over 10 models per arm and width. **(a)** The frontier tracks the code: training only the readout leaves $\kappa_{\min}$ where an untrained code puts it — below it, in fact — while training both lifts it at every width. **(b)** The same models placed by cross-talk against task loss. The horizontal axis is R_{readout} , so 1 is the best readout of the code that arm was given; the vertical axis is how much lower the trained readout’s own training loss is than that optimum’s. The frozen arm sits at nearly twice the cross-talk, which invites reading it as a failed optimisation, but it beats the cross-talk optimum on the loss it trained under, so the cross-talk is bought rather than lost. Its three widths coincide: neither coordinate moves as the problem grows. The jointly trained arm is better on both axes at once and pulls further ahead with width, so co-adaptation does not trade the two objectives against each other — it aligns them.

optimisation difficulty: the jointly trained arm attains $R_{\text{readout}} = 1.0185$ and a task advantage of 9.71 at $d = 200$, so it does not trade one against the other at all, while the frozen arm can only buy the second with the first. The joint arm’s task advantage also grows with width, 6.59 to 8.40 to 9.71, where the frozen arm’s is flat (4.39, 4.70, 4.50). Training the code appears to align the task objective with the cross-talk objective; freezing it leaves them in conflict that no amount of readout training resolves.

The frontier is now exact.. Earlier we reported $\kappa_{\min}$ over a subset of features and called it an upper bound. Evaluating every feature of all 180 models shows the bound was tight: the subset overestimated by 0.0206 on average at $d = 200$ and never by more than a tenth of a level. Its *justification* is weaker than its accuracy, though. The subset was chosen by Theorem 8.2, and that theorem is sufficient rather than necessary: on trained codes it locates the true argmin in only 6 of 20 models at $d = 200$, where the true argmin sits at leverage rank 38 by median, outside the window. On untrained codes it succeeds every time. Training pulls the lowest-leverage feature and the worst-frontier feature apart, and the numbers above are the exact ones.

A fourth width, committed to in advance. The frontier over $d \in \{50, 100, 200\}$ fits $\kappa_{\min} \propto d^{0.4202}$, which extrapolates to 10.759 at $d = 400$. We registered that number, then ran the width: 10 models per arm at $F = 2d = 800$, same protocol. The measurement is 10.510, an error of -2.31%. This is the only quantitative prediction the work is in a position to make and the only one tested out of sample, and we read it as a local rate rather than a law, because the exponent drifts downward across the four widths and Section 11.5 explains why four points cannot separate the candidates.

Two other things replicate at this width. The loss still decides the interface: L^2 reaches $R_{\text{geom}} = 32.3131$ with its frontier at 1.0653, against L^4 's 1.0001 and 10.5098, and the untrained code again sits near the floor at 1.0025 with a frontier of 8.9069. And the matched decoder comparison continues in the direction the smaller widths established, more sharply: at $d = 400$ the affine probe leads the network by -1.00 of a sparsity level in 10 of 10 models, where the three smaller widths gave a tie or a fraction of a level. The probe's advantage grows with width, which is the claim Section 11.4 makes and the opposite of the reading this work began with.

The margin over an untrained code is closing, and that is the finding of this width. Both frontiers grow with width, but not at the same rate: fitted over all four widths the trained code gives $d^{0.4101}$ and the untrained one $d^{0.4977}$ (Fig. 6a). The untrained exponent is the larger, so the advantage of training is a decreasing function of width in both readings of "advantage". As a ratio it falls throughout and faster each time: 1.4195, 1.3582, 1.2882, 1.1800. In absolute sparsity levels — the operationally meaningful unit, since the frontier licenses supports up to $\lceil \kappa_{\min} \rceil - 1$ — it rises to a peak at $d = 200$ and then falls: 1.3260, 1.5866, 1.7974, 1.6029. The fall is larger than the sampling noise: paired bootstrap over models gives a drop of 0.1940 with interval $[0.0903, 0.2818]$, and the two widths' intervals do not overlap. Taken at face value the two fitted laws meet near $d \approx 3039$.

We state this rather than the extrapolation. Four widths spanning a factor of eight cannot establish where two rates cross, and $d \approx 3039$ is well outside the range measured. What the data does support is narrower and still consequential: *training buys frontier at every width we measured, and the amount it buys stops growing by $d = 400$.* A reader entitled to conclude that gradient descent finds structurally better codes would want that margin to be at least stable in width, and here it is not. Whether the trained code's deceleration reflects a real ceiling or the fixed optimisation budget — 50 000

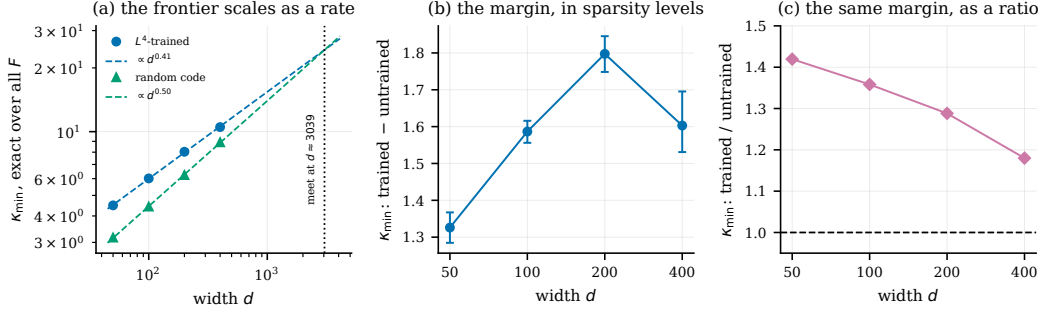


Figure 6: **The frontier’s rate, and the margin it puts in question.** All values are the exact $\kappa_{\min}$ over every one of the F features, never the subset. **(a)** Both codes’ frontiers follow a power law over the four widths, and the untrained exponent is the larger; the fitted lines meet near $d \approx 3039$, far outside the measured range, and the crossing is drawn to show what the rates imply rather than as a prediction. **(b)** The margin in sparsity levels, with paired bootstrap intervals over models; it peaks at $d = 200$ and the fall to $d = 400$ exceeds the sampling noise. **(c)** The same margin as a ratio, which falls at every step. The two panels are shown separately because they are the two honest readings of the same quantity and they do not agree until $d = 400$.

steps at every width, so the largest models get the least optimisation per parameter — this campaign cannot say, and it is the first thing a larger budget should test.

And the shortcut has stopped being defensible.. The same width shows the subset heuristic’s justification collapsing, which is why every number in the two paragraphs above is the exact minimum over all F features. The mean overestimate grows 0.0025, 0.0118, 0.0206, 0.0398; the median leverage rank of the true argmin grows 5, 23, 38, 131; and at $d = 400$ the subset locates the true argmin in 2 of 10 trained models. The error is still small in absolute terms, but it is one-sided and it is asymmetric between the arms: untrained codes keep the true argmin at leverage rank 1, so their subset value *is* exact, while trained codes drift. A margin computed from subset values is therefore biased in favour of training by an amount that grows with exactly the variable under study — which is why we computed the exact frontier for all four widths before making the claim above, and why we would not trust the shortcut past $d = 200$ in future work.

11.5. E6: a scalability stress test, not a scaling law

This campaign was designed to measure how the recovery threshold scales, and it does not have the resolution to do that. We report it for what it

does establish — that the measurement itself runs at a width and feature count where the code cannot be materialised — and withdraw the scaling-law reading.

Scaling to $d = 1\,024$ at $F = 1\,048\,576$ features lets the recovery threshold be traced over a factor of eight in width (Table 7, Fig. 7). On the grid, s_{95} grows from 1 at $d = 128$ to 8 at $d = 1\,024$.

That grid statistic needs care, because it floors the crossing and floors it unevenly. The true 95% crossing lies between grid points, and the gap is proportionally largest where the transition is sharp relative to the spacing — at $d = 128$ the interpolated crossing is 1.18 against a reported 1. Interpolating instead of flooring changes the headline: growth over the sweep falls from 8.00 to 6.89, against 5.60 for an exact $d/\ln d$ law. The floored figure therefore overstates the growth by roughly a third, purely as an artefact of the grid, and we quote the interpolated values from here on.

Four widths then cannot identify a law. On the interpolated crossings a one-parameter fit of $s_{95} = c d / \ln d$ gives $c = 0.0563$ with $R^2 = 0.9882$ over 4 points, but a plain linear law $c d$ reaches $R^2 = 0.9636$ and $c d / \ln^2 d$ reaches $R^2 = 0.9630$ on the same four points. The three are indistinguishable on this evidence, the base of the logarithm is not identifiable at all, and we therefore claim no scaling law — neither for $d/\ln d$ nor against it. A previous version of this work drew a $d/(16 \ln d)$ reference curve here; it has been withdrawn, because its constant is inherited from the earlier literature rather than derived anywhere in this work, and because it sat *above* the measured thresholds at every width and so was not the conservative reference its form suggests. Corollary 6.6 fixes only the shape $s \leq c d / \log d$, for an unspecified universal c .

A second qualification. s_{95} is a point estimate: at $d = 1\,024$ the campaign runs 50 trials, so even a flawless 50-for-50 result gives a Wilson lower bound of only 0.929. No sparsity at that width can be certified at the 95% level with 95% confidence on this budget; the reported thresholds are best estimates, and a claim at confidence would need more trials.

The whole campaign took 29.9 minutes on CPU, with $d = 1\,024$ accounting for 21.3 of them over 50 trials. Materialising the code at that size would have required 4 GB; the streaming implementation of Algorithm 3 keeps the peak footprint proportional to Bd .

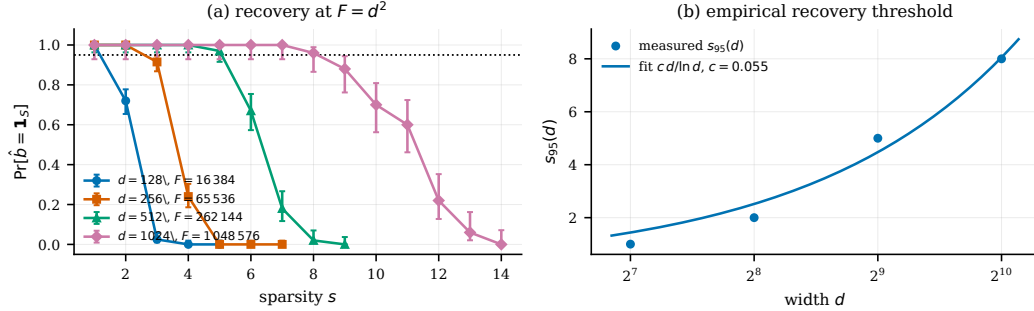


Figure 7: **Threshold recovery at quadratic feature load, up to $d = 1024$ ($F = 1\,048\,576$).** (a) Empirical exact-recovery probability with Wilson 95% intervals; a fresh random code is drawn per trial. (b) The empirical threshold $s_{95}(d)$ with the fitted $c d / \ln d$ curve shown as a fit only: four widths do not distinguish it from $c d$ or from $c d / \ln^2 d$, and no scaling law is claimed. Generated by `scripts/make_figures.py` from `results/e6`.

Table 7: E6: recovery threshold and measured cost by width.

d	$F = d^2$	trials	s_{95}	$d/(16 \ln d)$	wall clock (min)
128	16 384	200	1	1.65	1.5
256	65 536	200	2	2.89	2.2
512	262 144	100	5	5.13	4.9
1024	1 048 576	50	8	9.23	21.3

11.6. Cost and traceability

Every campaign ran on CPU alone; Table 8 reports the measured wall clock, which totals 343.8 minutes across the six. Table 9 closes the loop between the claims above and the artefacts behind them: for each experimental claim it names the figure or table that shows it, the raw result file the numbers come from, and the script and configuration that produce that file. Together with the automatic check that every measured number in this manuscript matches the saved results (Appendix Appendix A), this is what makes the reported quantities auditable rather than merely stated.

Table 8: Measured computational cost, read from the run records. E1–E6 are CPU-only and hide any accelerator that is present, so they cannot silently become irreproducible on a CPU-only machine. E7 and E8 train on a GPU; every number this paper draws from them is then recomputed from their saved weights on a CPU, so no claim here depends on access to an accelerator. *jobs* counts models for the GPU campaigns and configured jobs for the others. [†]Stage A was resumed, and a run record times only the final invocation, so this figure is the last leg rather than the campaign.

campaign	question	jobs	workers	device	wall clock (min)
E1	floor on random codes	–	1	CPU	0.6
E2	recovery at $F = d^2$	–	1	CPU	67.5
E3	average linear energy	–	1	CPU	0.2
E4	optimised codes	168	12	CPU	169.6
E5	trained toy model	25	6	CPU	76.1
E6	scaling to $d = 1024$	–	4	CPU	29.9
E7-A	trained-network grid	180	8	CUDA	306.1 [†]
E7-B	feature load and sparsity	100	10	CUDA	193.5
E7-C	$d = 400$ frontier rate	30	10	CUDA	103.8
E8	frozen code vs frozen readout	60	10	CUDA	77.0
E8	the same at $d = 200$	30	8	CUDA	123.2
total					1147.5

Table 9: Traceability: each experimental claim, the figure or table that shows it, the raw result file it comes from, and the script and configuration that produce it. Reproduction commands are in Appendix [Appendix A](#).

claim	figure / table	raw result file	script (config)
The floor is respected by every calibrated interface	Tab. 3 , Fig. 1a	<code>results/e1/raw/e1_raw_results</code>	<code>experiments/e1_welch_floor.py</code> (<code>configs/e1.json</code>)
Threshold recovery succeeds where linear readout cannot be exact	Fig. 1b	<code>results/e2/raw/e2_raw_results</code>	<code>experiments/e2_threshold_recovery.py</code> (<code>configs/e2.json</code>)
Average linear energy follows the s/d scale	Fig. 1c	<code>results/e3/raw/e3_raw_results</code>	<code>experiments/e3_linear_energy.py</code> (<code>configs/e3.json</code>)
Learned codes attain the floor	Tab. 4 , Fig. 2	<code>results/e4/raw/e4_raw_results</code>	<code>experiments/e4_optimized_codes.py</code> (<code>configs/e4.json</code>)
A trained network attains the floor and separates the two criteria	Tab. 5 , Fig. 3	<code>results/e5/raw/e5_raw_results</code>	<code>experiments/e5_trained_toy.py</code> (<code>configs/e5.json</code>)
$s_{95}(d)$ grows with the $d/\ln d$ scale	Tab. 7 , Fig. 7	<code>results/e6/raw/e6_raw_results</code>	<code>experiments/e6_threshold_scaling.py</code> (<code>configs/e6.json</code>)

12. Discussion

Three readings survive the campaign, and one that motivated the work does not.

The floor is a target, not an obstacle — and reaching it means less than it looks. E4 shows gradient descent under the unit-diagonal constraint converging to a unit-norm tight frame, and a network trained on a task with no interference objective lands within 1.0006 of the same bound. But an untrained i.i.d. code reaches $R_{\text{geom}} = 1.0187$ without any training at all, so proximity to the rank-trace floor is what the ensemble gives for free at this load rather than a signature of learned structure. What distinguishes models is not whether they approach the floor but whether the floor *binds* them, and there the separation is enormous: L^2 drives R_{geom} to 29.8734 at $d = 200$ and collapses its frontier to 1, while L^4 stays at 1.0002.

The nonlinearity does not out-decode an affine probe of the state it produces. Given the same post-ReLU representation and the same threshold treatment, the network and the best fitted affine probe tie on s_{95} in 119 of 120 trained models. We registered this outcome as a result before running, and it is one: for support recovery on this task, at these widths, the network’s second layer is doing what a linear probe of its first layer already does. The gap that earlier framings read as a nonlinear advantage is a gap between *representations* — the preactivation is more affinely decodable than what the ReLU leaves — and attributing it to the decoder confuses where the information went with who read it.

What training buys is the code. A random code with only its readout trained keeps an untrained code’s frontier, 4.122 against 4.373, while training both reaches 6.014. The frozen arm’s own decoder also ends at $R_{\text{readout}} = 1.9217$, near twice the closed-form cross-talk optimum for the code it was handed — but that is not a failed optimisation. It beats that optimum by 4.70 on the loss it was trained under, so on a fixed random code the task objective and the cross-talk objective genuinely diverge, and the readout is right to follow the former. What the joint arm gains is that it need not choose: it holds $R_{\text{readout}} = 1.0228$ while beating the same baseline by 8.40. Read that way, training the code aligns the two objectives, and that is a statement about what co-adaptation buys rather than about the difficulty of the conditional problem.

What does not survive is the reading this work began with: that the L^4 model’s thresholded output keeps working past where linear readouts of

its own code collapse. Against the fixed readouts of E5 that appeared true; against a probe fitted with the same care as the network was trained, it is not.

This gives the debate in [3, 4] an instrument. The question “does this network compute in superposition?” becomes three measurements: how far the code is from its own floor rather than from the ensemble’s, how far the network’s decision function is from an affine readout of the same state, and, per feature, the largest sparsity at which any affine rule could succeed at all. None requires reverse-engineering the mechanism, and the third is exact rather than empirical — which matters, because the campaign’s history is a series of empirical readings that reversed when the estimand or the baseline was tightened, while κ_i did not move.

For capacity theory, the interface distinction explains why the Hänni and Adler–Shavit results coexist rather than compete (Section 9). It also cautions against reading sparse-autoencoder dictionary sizes as computation capacities: those are reconstructive quantities, and the diagnostic makes explicit that reconstruction and recursive Boolean decoding are governed by different criteria.

12.1. Limitations

1. **One trained setting.** E5 covers a single toy task at $d = 50$, $F = 100$, with 5 seeds per condition. Nothing here is a claim about large trained language models.
2. **Out-of-distribution evaluation.** The diagnostic is defined on Boolean sparse states, while the toy models are trained on continuous sparse inputs. It therefore measures which interface the learned code *supports*, not what the network does on its native distribution.
3. **Constant sparsity in the application.** The Hänni-template comparison assumes $s = O(1)$, whereas the distributional results allow s up to $O(d/\log d)$; real features are continuous with heterogeneous sparsity.
4. **The floor bounds linear readouts only.** Nonlinear decoders and distribution-specific average-case mechanisms are outside its scope — which is the point of the separation, but also a limit on what the diagnostic can certify.
5. **Fit with few points.** The $cd/\ln d$ fit in E6 rests on 4 widths, on which a plain linear law fits nearly as well ($R^2 = 0.9636$ against 0.9882). The

data are consistent with the predicted shape and do not discriminate it from the alternatives; the logarithm’s base is not identifiable at all.

6. **s_{95} is a point estimate, not a certified threshold.** The trial budget shrinks with width, and at $d = 1\,024$ even a perfect run gives a Wilson lower bound of 0.929—below the 95% level being estimated. The reported thresholds are best estimates; certifying them at confidence would need more trials at the large widths.
7. **The linear side’s non-exactness is the theorem, not a measurement.** Wherever we report that a linear readout “cannot be exact”, that follows from Theorem 5.1 by construction for any rank- d unit-diagonal interface. What the experiments measure is the *magnitude* of that error for codes that were actually learned, and how far a threshold decoder gets before it bites.
8. **Correlated curve points.** The nested-support trick makes each point of $p_{\text{rec}}(s)$ marginally unbiased but correlates points within a trial; the Wilson intervals are per-point and should not be read as a joint confidence band.
9. **Open log gap.** The Adler–Shavit capacity bracket retains a single logarithmic factor of slack, which is not resolved here.
10. **The leverage prediction is a sufficient condition, and its selection heuristic decays with width.** Corollary 8.4 predicts the L^2 collapse from leverage alone and holds at all three widths of Section 11.4. But it is applied at $h_{\min}$, and it is sufficient rather than necessary, which shows up in where the true worst feature sits: the lowest-leverage window contains the true $\arg \min \kappa$ in 6 of 20 L^4 models at $d = 200$ and 2 of 10 at $d = 400$, where the true $\arg \min$ sits at leverage rank 131 by median — against every model of every untrained code, at every width. Training pulls the two apart, and the gap widens with width, so the theorem localises the failure less and less well as the code improves. Nothing reported depends on the heuristic — the frontiers quoted are exact — but a reader should not take low leverage as a reliable pointer to the binding feature, and past $d = 200$ should not use it as a sampling window at all.
11. **The exact frontier does not extrapolate much further.** Every κ_i reported here is exact — all F features of every model at all four widths, including the 10 models per arm at $d = 400$, $F = 800$. An earlier version of this limitation said the $d = 400$ sweep would be unaffordable and that a

fourth width would have to return to a subset estimate; it was run instead, in a few hours on ten CPU workers, and this entry is corrected rather than removed because the estimate was wrong in the direction that mattered. What remains true is the shape of the cost: one linear programme per feature, about 0.24s at $d = 200$, growing steeply in d while F grows with it. A fifth width would be a different proposition, and the honest statement is that the statistic is exact everywhere we report it and that we do not know where it stops being affordable.

12. **The nonlinear witness is built but not yet used.** The extractor of Section 8.1 is implemented and validated — every witness it returns is confirmed inseparable by a linear programme, and on small codes against brute-force enumeration — but two things are missing. The Radon compression to $d + 2$ states is not implemented, so the witnesses are valid and not minimal; and no result reported in this paper is derived from one, so the sharper claim it licenses is not made here.
13. **How much the ReLU costs is quantified at one width only.** Section 11.4 establishes that the pre-ReLU state is more affinely decodable than the post-ReLU one everywhere. How *much* more, in sparsity levels, is harder: κ is defined on Φb , and $\text{ReLU}(\Phi b)$ is not a linear image of the state polytope, so the post-ReLU side has to be sampled and tested rather than solved. Sampling can only refute separability, never confirm it, so it overestimates the frontier — by two to eight levels here, against the exact κ — and the overestimate is only harmless in a difference taken with the same sampler and the same draws on both sides. At $d = 50$ that difference is 0.87 levels $[0.63, 1.10]$ for L^4 and 2.03 $[1.77, 2.27]$ for an untrained code. At $d = 100$ and $d = 200$ the L^4 cells saturate the sampler’s ceiling on both sides, so their difference is censoring and not measurement, and we report no value there. A larger sampling budget would resolve it and we have not spent one.
14. **One control the analysis wants and does not have.** A random *tight frame*, whose R_{geom} is exactly 1 rather than the ≈ 1.01 of an i.i.d. code, would sharpen the claim that near-floor geometry is generic: as it stands the comparison is against a code that is close to the floor rather than on it, so a small residual advantage for the trained code could be conditioning rather than structure.
15. **The margin over an untrained code stops growing inside our**

own range. This is the sharpest limitation on the headline claim, and it comes from our own data rather than from something we did not run. Section 11.4 reports the trained frontier at $d^{0.4101}$ and the untrained one at $d^{0.4977}$: the untrained exponent is larger, the ratio falls at every step, and the margin in sparsity levels peaks at $d = 200$ and falls by 0.1940 [0.0903, 0.2818] at $d = 400$. “Training buys the code” therefore holds at every width we measured and is not established as a property that survives scale. Two candidate explanations are not separated here: a real ceiling on what the frontier can buy, or the fixed optimisation budget of 50 000 steps at every width, which gives the largest models the least optimisation per parameter. Distinguishing them needs a budget sweep at fixed width, which is cheap, and a fifth width, which is not.

12.2. Open problem

Corollary 5.10 shows that no reset staying inside the small-error ε -linear class can improve the worst-case cross-talk scale beyond $d^{-1/2}$ when $F \gg d$, while Theorem 6.3 shows thresholded recovery survives to $s = O(d/\log d)$ at quadratic load. Any interpolation beyond the Hänni template must therefore leave the small-error linear class. This makes one question concrete.

Conjecture 12.1 (Generic robust threshold reset). Fix $s = O(1)$. There are constants $c, C, K > 0$ such that for every d and every $F \leq c d^2 / \log^C d$ there exist a code $\Phi \in \mathbb{R}^{d \times F}$, an input readout R_{in} , a width- $\tilde{O}(d)$ reset module $\mathcal{R}_\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and a scoring map S such that, for every s -sparse b and every x with $\|R_{\text{in}}x - b\|_\infty \leq K d^{1/2} / (F^{1/2} s^{1/4})$, the output $z = \mathcal{R}_\Phi(x)$ is composable with the next stage and satisfies $b_i = 1 \Rightarrow (Sz)_i \geq 3/4$ and $b_i = 0 \Rightarrow (Sz)_i \leq 1/4$.

The incoming-error tolerance is the essential clause: a reset theorem for clean inputs $x = \Phi b$ alone would not support recursive composition. By Corollary 5.10 any proof must use an explicitly nonlinear or thresholded decoding mechanism, as Adler–Shavit do.

13. Conclusion

We proposed the Interface Diagnostic, a computable procedure that decides which decoding interface an overcomplete code or a trained network maintains, and how far it is from the best possible one. It rests on a rank-trace floor for unit-diagonal linear readouts, in a gain-normalised form that

bounds the interference-to-gain ratio, and on a separation between the linear and threshold criteria proved under one and the same sparse-state model. Its mean-square statistics run in $O(Fd^2)$ time and $O(d^2)$ memory and reach $F \approx 10^6$ features on a laptop CPU.

Two additions make the diagnosis about a code rather than about an ensemble. Computing a code’s own floor in closed form splits any attainment ratio into a geometry term and a decoder term, and the split is deflationary: under the calibrated pseudoinverse the decoder term is one by construction, so the ratio a reader is invited to read as a decoder result is nothing of the kind. The trained L^4 network’s 1.0006 and the L^2 baseline’s factor 28.8135 are statements about leverage profiles. And for support recovery, one linear programme per feature returns the largest sparsity at which any affine probe can detect that feature, with a certified margin when it succeeds and a checkable certificate when it fails. Leverage links the two: low leverage forces the affine frontier down, with no decoder in the argument, and an explicit code refutes the converse, so the hierarchy holds in one direction only.

Applied to 180 trained models, the instrument returns a negative result and two positive ones. The negative: given the same representation and the same threshold treatment, the network and a fitted affine probe tie on exact support recovery in 119 of 120 models, so on this task the nonlinearity does not out-decode a linear probe of the state it produces. The first positive: what separates models is the loss, not the architecture — L^2 pushes R_{geom} to 29.8734 and collapses its frontier to 1, while L^4 holds 1.0002 and a frontier of 8.0352. The second: that separation is bought by training the code and not the readout, since a frozen random code with a fully trained readout keeps an untrained code’s frontier.

Two things we decline to claim. There is no scaling law here: the threshold-recovery campaign reaches $d = 1\,024$ at about a million features, which establishes that the measurement scales, but four widths do not distinguish $c\,d/\ln d$ from the alternatives. And proximity to the rank–trace floor is not evidence of learned structure, because an untrained code already achieves it.

The broader lesson is that capacity questions for overcomplete representations are not settled by counting features, nor by measuring distance to a bound that random codes already meet. They depend on which decoding interface a particular code maintains, and on which representation is being read — both of which are now measurable, per feature and exactly, for the code in hand.

Acknowledgements

The authors thank the VRAIN institute and the Universitat Politècnica de València for institutional support, and two frame-theory researchers who, after reading the preprint, pointed out prior work containing the proof mechanism of Theorem 5.1 and raised the calibration question that became Corollary 5.2 and Remark 5.3.

Funding

H.B. acknowledges support from the UPV doctoral programme. No additional external funding was received for the submitted work.

CRedit authorship contribution statement

Hector Borobia: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Visualization, Writing – original draft. **Elies Seguí-Mas:** Supervision, Methodology, Writing – review & editing. **Guillermina Tormo-Carbó:** Supervision, Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No human-subject, biological, clinical or proprietary data were generated or analysed. All data underlying the figures and tables are synthetic, generated by the released code from recorded seeds, and are included as raw JSON result files in the public repository.

Code availability

All code, configurations, logs, raw results and the sources of this manuscript are publicly available at <https://github.com/hechoar/linear-readout-threshold-recovery>, together with a single entry point that reproduces every experiment, table and figure (Appendix Appendix A).

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used AI-assisted tools for exploratory mathematical checking, drafting support, language editing and literature-search assistance. After using these tools the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. AI tools are not listed as authors and did not determine the research questions, proof strategy or final claims. All mathematical statements, proofs, citations and comparisons with source papers were verified by the authors.

References

- [1] K. Hänni, J. Mendel, D. Vaintrob, L. Chan, Mathematical models of computation in superposition, version v1, 10 August 2024 (2024). [arXiv:2408.05451](#).
- [2] M. Adler, N. Shavit, On the complexity of neural computation in superposition, version v3, 26 February 2026 (2024). [arXiv:2409.15318](#).
- [3] J. Bhagat, S. Molas-Medina, G. Giguemiani, S. Heimersheim, Compressed computation is (probably) not computation in superposition (2026). [arXiv:2606.14673](#).
- [4] F. Ferreira da Silva, S. Heimersheim, Compressed computation under l^4 loss is likely computation in superposition (2026). [arXiv:2607.04800](#).
- [5] L. R. Welch, Lower bounds on the maximum cross correlation of signals, IEEE Transactions on Information Theory 20 (3) (1974) 397–399.
- [6] T. Strohmer, R. W. Heath, Grassmannian frames with applications to coding and communication, Applied and Computational Harmonic Analysis 14 (3) (2003) 257–275.
- [7] S. Waldron, Generalized Welch bound equality sequences are tight frames, IEEE Transactions on Information Theory 49 (9) (2003) 2307–2309.

- [8] S. Datta, S. D. Howard, D. Cochran, Geometry of the Welch bounds, *Linear Algebra and its Applications* 437 (10) (2012) 2455–2470, preprint [arXiv:0909.0206](#).
- [9] R. Aceska, M. Kaczanowski, Cross frame potential, *Numerical Functional Analysis and Optimization* 43 (14) (2022) 1663–1678, preprint [arXiv:2205.05613](#).
- [10] O. Christensen, S. Datta, R. Y. Kim, Equiangular frames and generalizations of the Welch bound to dual pairs of frames, *Linear and Multilinear Algebra* 68 (12) (2020) 2495–2505.
- [11] M. Adler, D. Alistarh, N. Shavit, Towards combinatorial interpretability of neural computation (2025). [arXiv:2504.08842](#).
- [12] L. Kong, I. Subramanian, Y. Shavit, M. Adler, D. Alistarh, N. Shavit, Expand neurons, not parameters (2025). [arXiv:2510.04500](#).
- [13] D. Braun, L. Bushnaq, S. Heimersheim, J. Mendel, L. Sharkey, Interpretability in parameter space: Minimizing mechanistic description length with attribution-based parameter decomposition (2025). [arXiv:2501.14926](#).
- [14] F. Ferreira da Silva, S. Heimersheim, Evidence for feature-specific error correction in LLMs (2026). [arXiv:2606.24964](#).
- [15] A. Klappenecker, M. Rötteler, Mutually unbiased bases are complex projective 2-designs, in: *Proceedings of the IEEE International Symposium on Information Theory (ISIT)*, 2005, pp. 1740–1744.
- [16] W. H. Kautz, R. C. Singleton, Nonrandom binary superimposed codes, *IEEE Transactions on Information Theory* 10 (4) (1964) 363–377. [doi:10.1109/TIT.1964.1053689](#).
- [17] D. L. Donoho, J. Tanner, Neighborliness of randomly projected simplices in high dimensions, *Proceedings of the National Academy of Sciences* 102 (27) (2005) 9452–9457. [doi:10.1073/pnas.0502258102](#).
- [18] A. d’Aspremont, L. El Ghaoui, Testing the nullspace property using semidefinite programming, *Mathematical Programming* 127 (1) (2011) 123–144. [doi:10.1007/s10107-010-0416-0](#).

- [19] H. Cunningham, A. Ewart, L. Riggs, R. Huben, L. Sharkey, Sparse autoencoders find highly interpretable features in language models (2023). [arXiv:2309.08600](#).
- [20] L. Gao, T. D. la Tour, H. Tillman, et al., Scaling and evaluating sparse autoencoders (2024). [arXiv:2406.04093](#).
- [21] T. Lieberum, S. Rajamanoharan, A. Conmy, et al., Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2 (2024). [arXiv:2408.05147](#).
- [22] E. J. Michaud, L. Gorton, T. McGrath, Understanding sparse autoencoder scaling in the presence of feature manifolds (2025). [arXiv:2509.02565](#).
- [23] L. Sharkey, B. Chughtai, J. Batson, et al., Open problems in mechanistic interpretability (2025). [arXiv:2501.16496](#).
- [24] J. Capon, High-resolution frequency-wavenumber spectrum analysis, *Proceedings of the IEEE* 57 (8) (1969) 1408–1418.
- [25] R. Vershynin, *High-Dimensional Probability: An Introduction with Applications in Data Science*, Cambridge University Press, 2018.
- [26] W. B. Johnson, J. Lindenstrauss, Extensions of Lipschitz mappings into a Hilbert space, in: *Contemporary Mathematics*, Vol. 26, American Mathematical Society, 1984, pp. 189–206.
- [27] H. Borobia, E. Seguí-Mas, G. Tormo-Carbó, How pruning reshapes features: Sparse autoencoder analysis of weight-pruned language models (2026). [arXiv:2603.25325](#).
- [28] Y. Liu, Z. Liu, J. Gore, Superposition yields robust neural scaling (2025). [arXiv:2505.10465](#).
- [29] N. Elhage, T. Hume, C. Olsson, et al., Toy models of superposition (2022). [arXiv:2209.10652](#).

Appendix A. Reproduction

From a clean checkout:

```
python -m venv .venv && . .venv/bin/activate      # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
python -m pytest -q tests/                        # 104 tests
bash scripts/run_all.sh                          # Windows: scripts\run_all.ps1
```

Individual campaigns, each of which writes `results/<id>/run_record.json` with the exact command, seeds, environment, duration and exit status:

```
python experiments/e1_welch_floor.py
python experiments/e2_threshold_recovery.py
python experiments/e3_linear_energy.py
python experiments/e4_optimized_codes.py
python experiments/e5_trained_toy.py
python experiments/e6_threshold_scaling.py
```

Adding `-smoke` to any of them runs a reduced configuration end to end in seconds. Figures, tables and the generated numbers are rebuilt with `scripts/make_figures.py`, `scripts/make_tables.py` and `scripts/make_numbers.py`; `scripts/check_manuscript_numbers.py` verifies that every measured number in this manuscript matches the saved results and exits non-zero otherwise. E6 writes results per width and per chunk and resumes an interrupted campaign without recomputing completed trials.

Appendix B. Verification of imported statements

Every statement imported from the two source frameworks was checked against the published version. Table [B.10](#) records the audit.

Appendix C. Auxiliary proofs

Appendix C.1. Sub-Gaussian interference

Let $u, u_1, \dots, u_m$ be independent uniform unit vectors in $\mathbb{R}^d$. Conditionally on u , each $X_j = \langle u, u_j \rangle$ is distributed as the first coordinate of a uniform point on S^{d-1} , hence has mean zero and is sub-Gaussian with variance proxy C_0/d for a universal C_0 [25, Thm. 3.4.6]. The X_j are conditionally independent, so their sum is sub-Gaussian with proxy $C_0 m/d$ and $\Pr\{|\sum_j X_j| > t\} \leq 2 \exp(-c_0 d t^2/m)$ with $c_0 = 1/(2C_0)$. This is the estimate used in Theorem [6.3](#).

Appendix C.2. The Bernoulli variant of the energy floor

Under $b_i \sim \text{Bernoulli}(p)$ independently, $\mathbb{E}[bb^\top] = p(1-p)I_F + p^2\mathbf{1}\mathbf{1}^\top$, so $\mathbb{E}_b \|Ab\|^2 = p(1-p) \|A\|_F^2 + p^2 \|A\mathbf{1}\|^2 \geq p(1-p)F(F-d)/d$ by Theorem 5.1. With $p = s/F$ and $s \leq F/2$ this gives $\frac{1}{F}\mathbb{E}_b \|Ab\|^2 \geq \frac{s(F-d)}{2dF} = \Omega(s/d)$, matching Proposition 6.5 up to the $(F-1)$ versus F denominator. Both variants are computed by the released code and both are reported in E3.

Appendix D. The nuclear-norm inequality

For $M \in \mathbb{R}^{F \times F}$ with singular value decomposition $M = \sum_{k=1}^r \sigma_k u_k v_k^\top$,

$$|\text{tr}(M)| = \left| \sum_k \sigma_k u_k^\top v_k \right| \leq \sum_k \sigma_k |u_k^\top v_k| \leq \sum_k \sigma_k = \|M\|_*,$$

and by Cauchy–Schwarz on $(\sigma_1, \dots, \sigma_r)$, $\|M\|_*^2 \leq r \sum_k \sigma_k^2 = r \|M\|_F^2$. With $r \leq d$ this gives $|\text{tr}(M)|^2 \leq d \|M\|_F^2$, as used in Theorem 5.1. Note that the identity $\text{tr}(M) = \sum_k \sigma_k$ holds only for positive semidefinite matrices and is *not* used.

Appendix E. Reference scales

The scales appearing in this literature count different objects and should not be read as a capacity ordering. Writing $F_{\text{CS}}(d, \alpha) = d g(\alpha)$ with $g(\alpha) = 1/((1-\alpha) \ln(1/(1-\alpha)))$ for compressed-sensing-style linearly decodable storage, $F_{\text{cert}}^{\text{H}}(d) = \tilde{\Theta}(d^{3/2})$ for the Hänni template certificate, $L_{\text{AS}}(d) = d^2 / \log^2 d$ and $U_{\text{AS}}(d) = d^2 / \log d$ for the two sides of the Adler–Shavit bracket, and $N_{\text{JL}}(d; \varepsilon) = \exp(\Theta(d\varepsilon^2))$ for Johnson–Lindenstrauss packing [26], one has

$$F_{\text{CS}} \ll F_{\text{cert}}^{\text{H}} \ll L_{\text{AS}} \lesssim F_{\text{rec}}^{\text{AS}} \lesssim U_{\text{AS}} \ll N_{\text{JL}}$$

as $d \rightarrow \infty$. This does not mean that recursive computation beats storage: N_{JL} counts passively packable states, F_{CS} counts linearly decodable features through a compressed bottleneck, and the middle two count recursively computable Boolean features under two different published frameworks. Michaud et al. [22] give a complementary reason why an observed dictionary size may fall below the template scale: if feature manifolds consume capacity, the relevant comparison is the effective dimension rather than the raw feature count.

Appendix F. Sparse-autoencoder context

As background only, our companion study [27] trains TopK sparse autoencoders ($k = 64$, expansion $8d$) on three language models and observes no dead features at $F = 8d$, placing those dictionaries well below both the $d^{3/2}$ template scale and the near-quadratic Adler–Shavit scale. Sparse-autoencoder dictionary size is a reconstructive quantity and is not a measurement of recursive computation capacity; these numbers are not used in any proof or experiment of the present paper, and are mentioned only to indicate where current empirical dictionaries sit relative to the scales of Appendix [Appendix E](#). Liu et al. [28] and Elhage et al. [29] provide further context on why strong superposition regimes are of interest.

Table B.10: Audit of imported statements.

source	as printed there	as used here
[1, Thm. 11]	targeted superpositional AND, precision $\varepsilon_{\text{in}} \rightarrow \varepsilon_{\text{out}}$ with $\varepsilon_{\text{out}} = \tilde{O}(\sqrt{s^2/d})$	Lemma 9.1, $\varepsilon_{\text{comp}}(d) = \tilde{O}(d^{-1/2})$ at $s = O(1)$
[1, Thm. 14]	$\varepsilon^{(1)} = \tilde{O}(\max\{s\mu, \sqrt{s\varepsilon}, \sqrt{s/d}\})$, $\mu^{(1)} = \tilde{O}(\max\{\sqrt{1/d}, \mu\})$	quoted in Sec. 9 only to note it does not by itself preserve the $\tilde{O}(d^{-1/2})$ invariant
[1, Cor. 15]	relaxes the near-sphere hypothesis of Thm. 14 at the cost of depth $1 \rightarrow 3$	not used for a capacity claim
[1, Thm. 21]	tolerance $\varepsilon < K d^{1/4}/(m^{1/2}s^{1/4})$, output $\varepsilon^{(1)} = O(\log(d)\sqrt{s}/\sqrt{d})$, $\mu^{(1)} = \tilde{O}(\sqrt{s}/\sqrt{d})$	Lemma 9.2 with m in the role of F
[1, Thm. 8]	width $d = \tilde{O}(m^{2/3}s^2)$, depth $2L$	Proposition 9.3, inverted at $s = O(1)$
[2]	$\Omega(\sqrt{m'} \log m')$ neurons and $\Omega(m' \log m')$ parameters; construction with $O(\sqrt{m'} \log m')$ neurons; at most $O(n^2/\log n)$ features with n neurons	Sec. 9; the parameter-to-neuron conversion is flagged as an architectural assumption of that model
[6, Thm. 2.3]	$\max_{i \neq j} \langle \phi_i, \phi_j \rangle \geq \frac{\sqrt{(F-d)/(d(F-1))}}{\sqrt{(F-d)/(d(F-1))}}$ for unit-norm systems	the Gram case of Theorem 5.1; the reduction is noted in Sec. 2.3